

claude-windows / fa4dc4eb-4705-47fd-b044-4f017b586b15

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\fa4dc4eb-4705-47fd-b044-4f017b586b15\subagents\workflows\wf_ac0cbb29-6bf\agent-a615adbd3c6527415.jsonl`
- SHA-256: `9fdf5f1a164e8a84abd75f2696f645615370f268a72a7bf43776ecce0131be9d`
- Source modified: `2026-06-19T15:49:04+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf_ac0cbb29-6bf`
- session_id: `fa4dc4eb-4705-47fd-b044-4f017b586b15`
```

Transcript

1. user (2026-06-19T15:46:34.345Z)

You are auditing a badminton rally-detection codebase for DOC/COMMENT STALENESS ? statements that CONTRADICT the current code or data.

HARD RULES:

- Report ONLY factual staleness with CODE EVIDENCE (file:line or symbol) that proves it. Open the cited code and confirm before reporting.
- NEVER invent a fact or a status. If a thing is built-but-default-OFF / unwired / unmeasured, say EXACTLY that ? do not overclaim "in production".
- A "[design]" tag is stale ONLY if the thing is actually BUILT in code (cite the symbol).
- DO NOT propose subjective rewrites, style/tone changes, or improvements. DO NOT relitigate owner-gated DECISIONS (owned-model roadmap, licensing/MIT-Apache stance, product/strategy choices, genuinely-still-design proposals).
- Preserve the document's voice and formatting; make the SMALLEST surgical edit. stale_text MUST be copied verbatim with enough context to be unique.
- IGNORE docs/archives/** (terminal snapshots) and any memory/ file.

Working dir is the repo root; use Read/Grep/Glob freely.

YOUR SLICE ? signals:

RALLY SIGNAL/FEATURE build-status staleness. Files: docs/MULTI_SIGNAL_FUSION_PLAN.md, docs/RALLY_DETECTION_GAP_CLOSURE.md, docs/ANALYZERS_AND_RECONCILERS.md, docs/HOW_RALLY_DETECTION_WORKS.md, docs/ABLATION_LAYER.md. For EVERY signal (WASB shuttle, person cue, Gate A net-crossings, Gate B occupancy/min_players, audio, shuttle_player_colocation / held-shuttle, shuttle-state, windowing R2/R3/R4/R5) reconcile the doc claimed status ([design]/built/wired/served/measured) against the ACTUAL code: backend/pipeline/detectors/{fusion_features.py,person_detector.py,rally_gate.py}, backend/pipeline/segmenters/{fusion_hybrid.py,yolo_hybrid.py}, backend/eval/{ablation.py,experiment.py>windowing.py}, backend/config/models.py. SEED (verify it): shuttle_player_colocation IS BUILT (backend/pipeline/detectors/fusion_features.py:168 + PERSON_FEATURE_NAMES) and wired in fusion_hybrid.py under cue_flags.person, but it is default-OFF, mock-tested, and UNMEASURED on golden ? so any doc tagging it purely "[design]"/"NEW (needs the person cue)"/"not built" is stale and should instead say the feature is built (fusion_features.py) but default-OFF + unmeasured on real data.

Return EVERY code-contradicting staleness item you can verify in your slice. Exhaustive within the slice; facts only.

2. assistant (2026-06-19T15:46:38.651Z)

I'll audit the signal/feature build-status claims in my assigned docs against the actual code. Let me start by reading the key code files and docs in parallel.

3. user (2026-06-19T15:46:39.140Z)

```
1      """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
```

```

2
3 NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4 math is unit-testable in a GPU-free dev container. The detection (person boxes,
5 shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6 module only turns those signals into a handful of **scale/translation-invariant**
7 features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8 memorise this court/camera and fail to generalise off a small labelled set).
9
10 Conventions:
11 - A person box is `(x1, y1, x2, y2)` in **pixels**; `(track_id, box)` when motion
needs it.
12 - **per_frame** is `{frame_idx: [(track_id, box), ...]}` keyed by the ORIGINAL video
frame
13 index (so it aligns directly with the shuttle trajectory's **frame** ? no clip re-
timing).
14 - A shuttle point has **frame**, **visible**, **x**, **y** (pixels) ? the WASB
trajectory shape.
15 - **court_polygon** is normalised 0-1 **[(x, y), ...]** or **None** (None ? no mask,
every
16 detection counts ? the player-count-only mode).
17 - **net** is **('axis 'x'|'y', position 0-1 normalised)** or **None** (None ? two-sided =
0.0).
18
19 Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20 floats in [0, 1] except **mean_player_motion** (a non-negative normalised speed).
21 """
22 from __future__ import annotations
23
24 from typing import Dict, List, Optional, Sequence, Tuple
25
26 BBox = Tuple[float, float, float, float]
27 Net = Tuple[str, float]
28
29
30 def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31     """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32     (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33     if polygon is None or len(polygon) < 3:
34         return False
35     x, y = point
36     inside = False
37     n = len(polygon)
38     j = n - 1
39     for i in range(n):
40         xi, yi = polygon[i]
41         xj, yj = polygon[j]
42         # Does the horizontal ray at y cross edge (i, j)?
43         if (yi > y) != (yj > y):
44             x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45             if x < x_cross:
46                 inside = not inside
47         j = i
48     return inside
49
50
51 def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
52     """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53     the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54     if frame_width <= 0 or frame_height <= 0:
55         return (0.0, 0.0)
56     x1, _y1, x2, y2 = box
57     return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58

```

```

59
60 def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
61     if frame_width <= 0 or frame_height <= 0:
62         return (0.0, 0.0)
63     x1, y1, x2, y2 = box
64     return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)
65
66
67 def person_in_court(box: BBox, frame_width: float, frame_height: float,
68                     court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
69     """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
70     True (count everyone) when it is None. Unusable frame dims (w/h <= 0) make the foot
71     point meaningless, so a masked check returns False rather than testing a bogus
72     (0,0)."""
73     if court_polygon is None:
74         return True
75     if frame_width <= 0 or frame_height <= 0:
76         return False
77     return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)
78
79 def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
80                    frame_height: float,
81                    court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
82     """Per-frame count of in-court persons (one entry per sampled frame)."""
83     return [
84         sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
85 court_polygon))
86         for _frame_idx, dets in sorted(per_frame.items())
87     ]
88
89 def _median(vals: Sequence[float]) -> float:
90     if not vals:
91         return 0.0
92     s = sorted(vals)
93     n = len(s)
94     return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))
95
96
97 def players_in_court(counts: Sequence[int]) -> float:
98     """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
99     return _median(list(counts))
100
101
102 def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
103     """Fraction of sampled frames with >= `min_players` in court (track stability vs
104 flicker)."""
105     if not counts:
106         return 0.0
107     return sum(1 for c in counts if c >= min_players) / len(counts)
108
109 def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
110                       frame_width: float, frame_height: float) -> float:
111     """Mean per-track centre displacement between consecutive sampled frames, with x
112     normalised by frame width and y by frame height (per-axis; uniform-scale-invariant,
113     not aspect-ratio-isotropic). 0.0 if <2 frames or no shared tracks."""
114     if frame_width <= 0 or len(per_frame) < 2:
115         return 0.0
116     frames = sorted(per_frame.keys())
117     steps: List[float] = []
118     for a, b in zip(frames, frames[1:]):

```

```

119         prev = {tid: box for tid, box in per_frame[a]}
120         for tid, box in per_frame[b]:
121             if tid in prev:
122                 cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
123                 cx1, cy1 = _center_norm(box, frame_width, frame_height)
124                 steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
125     if not steps:
126         return 0.0
127     return sum(steps) / len(steps)
128
129
130 def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
131                     frame_height: float, net: Optional[Net],
132                     court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
133     """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
134
135     A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
136     The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
137     0.0.
138     """
139     if net is None or not per_frame:
140         return 0.0
141     axis, pos = net
142     both = 0
143     for _frame_idx, dets in per_frame.items():
144         near, far = False, False
145         for _tid, box in dets:
146             if not person_in_court(box, frame_width, frame_height, court_polygon):
147                 continue
148             fx, fy = _foot_norm(box, frame_width, frame_height)
149             coord = fx if axis == "x" else fy
150             if coord < pos:
151                 near = True
152             else:
153                 far = True
154             if near and far:
155                 both += 1
156     return both / len(per_frame)
157
158 def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
159                   pad_frac: float) -> bool:
160     """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
161     width/height (the 'adjacent / in-hand' tolerance)?"""
162     x1, y1, x2, y2 = box
163     px = pad_frac * frame_width
164     py = pad_frac * frame_height
165     return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)
166
167
168 def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
169                             frame_width: float, frame_height: float,
170                             pad_frac: float = 0.03) -> float:
171     """Fraction of (visible shuttle frames that were also person-sampled) in which the
172     shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
173     ~0**
174     ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
175     """
176     if not per_frame or frame_width <= 0 or frame_height <= 0:
177         return 0.0
178     aligned = 0
179     colocated = 0
180     for p in shuttle_points or []:

```

```

180         if not getattr(p, "visible", False):
181             continue
182         dets = per_frame.get(int(p.frame))
183         if dets is None:             # shuttle frame wasn't person-sampled ? can't judge
184             continue
185         aligned += 1
186         if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
187             for _tid, box in dets):
188             colocated += 1
189     return colocated / aligned if aligned else 0.0
190
191
192     # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
193     # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
194     PERSON_FEATURE_NAMES = [
195         "players_in_court", "two_sided_motion", "mean_player_motion",
196         "in_court_ratio", "shuttle_player_colocation",
197     ]
198
199
200     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
201 shuttle_points,
202                                     frame_width: float, frame_height: float,
203                                     court_polygon: Optional[Sequence[Tuple[float, float]]] =
204 None,
205                                     net: Optional[Net] = None,
206                                     min_players: int = 2) -> Dict[str, float]:
207     """Aggregate the per-window person features from per-frame detections + the in-
208 window
209 shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
210     counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
211     return {
212         "players_in_court": players_in_court(counts),
213         "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
214 court_polygon),
215         "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
216         "in_court_ratio": in_court_ratio(counts, min_players),
217         "shuttle_player_colocation": shuttle_player_colocation(
218             per_frame, shuttle_points, frame_width, frame_height),
219     }

```

4. user (2026-06-19T15:46:40.104Z)

```

1     """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3     WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4     the opponent for service after losing a point, that single trajectory looks like a
5     rally. The discriminator is structural, not duration-based: a real rally has the
6     shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7     single service feed crosses ~once.
8
9     This module is camera-agnostic and needs NO new model ? it works on the trajectory
10    we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11    axis with the larger shuttle spread) and the net position (median shuttle coord on
12    that axis ? the shuttle clusters at the two ends and crosses the net region often,
13    so the median sits near the net), then counts sign changes of (coord - net) across
14    the visible points inside a candidate window, with a deadband to ignore jitter.
15
16    Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17    Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18    windows from trajectory_to_action_windows, or to fold into it later.
19    """
20    from __future__ import annotations
21

```

```

22 from dataclasses import dataclass
23 from typing import List, Optional, Sequence, Tuple
24
25 Interval = Tuple[float, float]
26
27
28 @dataclass
29 class GatedWindow:
30     start: float
31     end: float
32     crossings: int
33     visible_points: int
34     kept: bool
35
36
37 def _spread(vals: Sequence[float]) -> float:
38     """Robust spread = p95 - p5 (ignores a few outlier detections)."""
39     if len(vals) < 2:
40         return 0.0
41     s = sorted(vals)
42     lo = s[max(0, int(0.05 * (len(s) - 1)))]
43     hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
44     return hi - lo
45
46
47 def _median(vals: Sequence[float]) -> float:
48     if not vals:
49         return 0.0
50     s = sorted(vals)
51     n = len(s)
52     return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])
53
54
55 def estimate_play_axis(points) -> Tuple[str, float, float]:
56     """Return (axis 'x'|'y', net_value, span) from visible points.
57
58     Play axis = the image axis along which the shuttle travels most (larger spread).
59     For a baseline camera that's Y (players top/bottom); for a side camera, X.
60     """
61     xs = [p.x for p in points if p.visible]
62     ys = [p.y for p in points if p.visible]
63     sx, sy = _spread(xs), _spread(ys)
64     if sx >= sy:
65         return "x", _median(xs), sx
66     return "y", _median(ys), sy
67
68
69 def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
70     """Count sign changes of (coord - net) across visible points, ignoring the
71     deadband zone near the net (so jitter at the net doesn't inflate the count)."""
72     crossings = 0
73     last_side = None
74     for p in window_points:
75         if not p.visible:
76             continue
77         v = (p.x if axis == "x" else p.y) - net
78         if abs(v) < deadband:
79             continue
80         side = v > 0
81         if last_side is not None and side != last_side:
82             crossings += 1
83         last_side = side
84     return crossings
85
86

```

```

87     def _points_in(points, fps: float, start: float, end: float):
88         return [p for p in points if start * fps <= p.frame <= end * fps]
89
90
91     def gate_windows(points, windows: List[Interval], fps: float,
92                     min_crossings: int = 2, deadband_frac: float = 0.06,
93                     estimate: Optional[Tuple[str, float, float]] = None) ->
List[GatedWindow]:
94         """Annotate each (start,end) window with its shuttle net-crossing count and a
95         keep decision (crossings >= min_crossings). Deadband is a fraction of the play
96         axis span. Net/axis are estimated once over the whole trajectory (stable).
97
98         `estimate` lets a caller scoring MANY windows off one trajectory pass a precomputed
99         `(axis, net, span)` (from `estimate_play_axis`) so the O(P log P) estimate isn't
redone
100         per window ? same result, avoids the per-window re-estimation."""
101         axis, net, span = estimate if estimate is not None else estimate_play_axis(points)
102         deadband = deadband_frac * span
103         out: List[GatedWindow] = []
104         for (s, e) in windows:
105             wp = _points_in(points, fps, s, e)
106             c = count_net_crossings(wp, axis, net, deadband)
107             vis = sum(1 for p in wp if p.visible)
108             out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
109                                   kept=c >= min_crossings))
110         return out
111
112
113     def apply_gate(points, windows: List[Interval], fps: float,
114                   min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
115         """Convenience: return only the windows that pass the crossing gate."""
116         return [(g.start, g.end) for g in
117                 gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
118

```

5. user (2026-06-19T15:46:40.609Z)

```

1     """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3     Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4     TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5     do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6     shuttle-seeded window via two overridable seams the engine exposes:
7
8     - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9     GPU-free, computed from the trajectory we already have); and, only when the person cue
10    is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11    (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
12    fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13    torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14    model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15    via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16    - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17    (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18    dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19    **Persisted candidates remain the full superset** regardless, so the offline
experiment
20    harness can re-score any variant.
21
22    Defaults reproduce the substrate exactly: net_crossings/person features are persisted

```

```

but
23     nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24     and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25     segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26     experiment harness (EXPERIMENT_HARNESS.md), never silently.
27     """
28     from typing import Any, Dict, List, Optional, Tuple
29
30     from backend.pipeline.segmenters.base import segmenter_registry
31     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
32     from backend.pipeline.detectors.base import DetectorRunner
33     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
34     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
35     from backend.pipeline.detectors import rally_gate
36     from backend.pipeline.detectors import fusion_features as ff
37
38
39     class FusionSegmenter(TrajectoryHybridSegmenter):
40         """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
41
42         family = "fusion"
43         display_label = "Fusion"
44
45         def __init__(self, db: Any, config: Any):
46             super().__init__(db, config)
47             # Built lazily only if the person cue is enabled (no detector model loaded
otherwise).
48             self._person: Optional[Any] = None
49             # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
estimate it
50             # over the whole trajectory once per candidate window (it depends only on
`points`).
51             self._axis_cache_key: Optional[int] = None
52             self._axis_cache: Optional[tuple] = None
53
54             @property
55             def name(self) -> str:
56                 return "fusion_hybrid"
57
58             @property
59             def description(self) -> str:
60                 return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
61                         "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
62                         "AI provider sets semantic boundaries.")
63
64             def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
65                 substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
66                 if substrate == "tracknet":
67                     cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
68                     return TrackNetRunner(cfg), {
69                         "weights_path": cfg.weights_path,
70                         "queue_length": cfg.queue_length,
71                         "fusion_substrate": "tracknet",
72                     }
73                 wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
74                 return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
75

```

```

76         def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
77                                     frame_width: float, video_path: str,
78                                     idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
79             stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
80             # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
81             # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
82             # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
83             gated = rally_gate.gate_windows(points, [[cw["start"], cw["end"]]], fps,
84                                                  min_crossings=0,
estimate=self._axis_estimate(points))
85             stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
86
87             fcfg = idx_cfg.get("fusion", {})
88             if fcfg.get("cue_flags", {}).get("person", False):
89                 stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
90             return stats
91
92             # P3 (learned fusion) first-step landing note (per MULTI_SIGNAL_FUSION_PLAN +
NEXT_STEPS 2026-06-14):
93             # Person features (from _person_features when cue_flags.person) are now persisted
for the harness
94             # `compare` LOVO litmus on the 6 golden videos. Eager-person-compute optimisation
(compute only
95             # for shuttle-seeded windows + padding, not whole video) is already in place via the
lazy
96             # self._person + windowed detections_per_frame. Next: actual harness run + lift
measurement
97             # (owner/hardware) before any served promotion. Flag-gated (default person OFF). See
98             # docs/NEXT_STEPS.md "Rally-detection quality track" and EXPERIMENT_HARNESS.md for
discipline.
99
100         def _axis_estimate(self, points: List[Any]) -> tuple:
101             """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
102             computed once per video, not once per candidate window (it depends only on
points)."""
103             key = id(points)
104             if self._axis_cache_key != key or self._axis_cache is None:
105                 self._axis_cache_key = key
106                 self._axis_cache = rally_gate.estimate_play_axis(points)
107             est = self._axis_cache
108             assert est is not None
109             return est
110
111         def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
112                             frame_width: float, video_path: str,
113                             fcfg: Dict[str, Any]) -> Dict[str, float]:
114             """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
115             frame range and compute the scale/translation-invariant person features. Lazily
builds
116             ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
117             # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
118             from backend.pipeline.detectors.person_detector import PersonDetector
119             if self._person is None:
120                 self._person = PersonDetector(self.config)
121
122             frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)

```

```

123         # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
124         # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
125         start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
126         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
127         fw = det["frame_width"] or frame_width
128         fh = det["frame_height"]
129         # If we can't trust the frame dims, every normalised feature would silently
collapse
130         # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
131         if fw <= 0 or fh <= 0:
132             return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
133
134         poly = fcfg.get("court_polygon")
135         polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
136         net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
137         shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
138         return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
139                                         court_polygon=polygon, net=net)
140
141     def _select_for_handoff(self, windows: List[Dict[str, Any]],
142                           window_stats: List[Dict[str, Any]],
143                           idx_cfg: Dict[str, Any]) -> List[int]:
144         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
145         (0) keep every window ? identical to the substrate. Persisted candidates are
146         untouched (full superset).
147
148         NEVER-EMPTY safeguard: a video that had ?1 window must keep ?1 after gating. If
the
149         thresholds would drop EVERY window (e.g. a short, noisy or one-sided-feed
trajectory
150         where no segment clears ``min_crossings``), the gate falls back to the full
ungated
151         set rather than emit a video with zero rallies. This makes Gate A/B strictly
152         *pruning* ? it can shrink a candidate set but never zero it out ? so layering it
on
153         the substrate is "do no harm": the bounded-windowing / largest-internal-gap
split
154         below it always wins on an otherwise-empty video. Observed on real footage
155         (mahadevpura-1): a single mega-window split into pieces, none clearing the
crossings
156         gate ? without this guard the whole clip yielded 0 rallies (no reel)."""
157         fcfg = idx_cfg.get("fusion", {})
158         min_crossings = int(fcfg.get("min_crossings", 0))
159         min_players = int(fcfg.get("min_players", 0))
160         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
161
162         keep: List[int] = []
163         for i, st in enumerate(window_stats):
164             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
165                 continue
166             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
167                 continue
168             keep.append(i)
169
170         if not keep and window_stats:
171             # Gating emptied a non-empty video ? fall back to the ungated set.
172             return list(range(len(window_stats)))
173         return keep
174
175     segmenter_registry.register(FusionSegmenter)
176

```

6. user (2026-06-19T15:46:40.914Z)

```

1      """Pydantic models for the application configuration.
2
3      This module defines the canonical, validated, typed configuration for the
4      Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6      The models support graceful migration from old config.json files via extra="allow".
7      """
8
9      from __future__ import annotations
10
11     from typing import Any, List, Literal, Optional
12
13     from pydantic import BaseModel, Field, field_validator, model_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18     class ValidationRelevanceConfig(BaseModel):
19         court_green_lower: list[int] = Field(default=[35, 40, 40])
20         court_green_upper: list[int] = Field(default=[85, 255, 255])
21         court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24     class ValidationConfig(BaseModel):
25         min_resolution_width: int = 1280
26         min_resolution_height: int = 720
27         min_fps: float = 29.9
28         min_blur_threshold: float = 20.0
29         max_scene_cuts_per_minute: float = 0.5
30         relevance: ValidationRelevanceConfig =
31         Field(default_factory=ValidationRelevanceConfig)
32
33     class TrackNetConfig(BaseModel):
34         wsl_distro: str = "Ubuntu"
35         repo_dir: str = "~/models/TrackNetV4"
36         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37         conda_env: str = "TrackNetV4"
38         weights_path: str = ""
39         queue_length: int = 5
40         stage_in_wsl: bool = True
41         wsl_stage_dir: str = "~/clips"
42         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43         velocity_threshold: float = 0.02
44         # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45         # decoded frame cache) are deleted automatically ? they are pure intermediates,
46         # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47         # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48         keep_frames: bool = False
49         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50         # 0 = disabled.
51         min_free_gb: float = 0.0
52
53
54     class WasbIndexConfig(BaseModel):
55         """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57         Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58         these knobs actually take effect ? previously this whole block was silently dropped
59         because nested config sections defaulted to extra='ignore'.
60         """

```

```

61     wsl_distro: str = "Ubuntu"
62     repo_dir: str = "~/models/WASB-SBDT"
63     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64     conda_env: str = "wasb"
65     weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66     sport: str = "badminton"
67     wsl_stage_dir: str = "~/clips"
68     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69     velocity_threshold: float = 0.02
70     # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
71     # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
72     # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
73     keep_frames: bool = False
74     # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
75     min_free_gb: float = 0.0
76     # FAST PATH (default OFF until owner-verified side-by-side): decode the staged video
77     # on the fly and feed frames straight to the detector, skipping the slow per-frame
PNG
78     # extraction that dominates a long clip (~46k PNGs for a 12-min 1080p60 video, which
79     # overran the 30-min timeout). Output is bit-identical to the PNG path (PNG is
lossless),
80     # so this is purely a speed/disk win; kept opt-in until the side-by-side confirms
it.
81     stream_video: bool = False
82
83
84     class FusionConfig(BaseModel):
85         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
86
87         Every default reproduces control behaviour: the fusion segmenter persists the
88         shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
89         true ? the person-cue features, but it does NOT gate the served handoff unless a
90         threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
91         is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
92         supplied ? off-court persons (spectators / adjacent courts) are excluded.
93         """
94         # Which trajectory substrate seeds the windows (shuttle stays primary).
95         substrate: Literal["wasb", "tracknet"] = "wasb"
96         # Windowing velocity threshold for the fusion family (the engine reads
97         # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
98         # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
99         velocity_threshold: float = 0.02
100        # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
101        # enabled ? so the default path never imports torch / touches the GPU.
102        cue_flags: dict[str, bool] = Field(default_factory=dict)
103        # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
104        # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
105        min_crossings: int = Field(default=0, ge=0)
106        # Served Gate B: when the person cue is on, drop windows with median in-court
players
107        # < this. 0 = OFF.
108        min_players: int = Field(default=0, ge=0)
109        # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
110        court_polygon: Optional[list[list[float]]] = None
111        # Net line for the person two-sided-motion split (person features only ? the shuttle

```

```

112     # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
113     net_axis: Literal["x", "y"] = "y"
114     net_position: float = Field(default=0.5, ge=0.0, le=1.0)
115
116
117     class IndexingConfig(BaseModel):
118         default_segmenter: str = "yolo_hybrid"
119         use_gpu: bool = True
120         motion_threshold: float = 0.08
121         min_segment_duration: float = 3.0
122         dead_time_merge_gap: float = 2.0
123         chunk_duration: float = 90.0
124         chunk_overlap: float = 10.0
125         detector_model: str = "fasterrcnn_resnet50_fpn"
126         detector_score_threshold: float = 0.5
127         yolo_velocity_threshold: float = 0.15
128         yolo_frame_skip: int = 3
129         yolo_tracker: str = "bytetrack.yaml"
130         yolo_prefetch_workers: int = 2
131         min_action_window_duration: float = 2.0
132         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
133         # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
134         # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
135         # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
136         # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
137         # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
138         # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
139         max_window_duration: float = 6.0
140         window_min_split_gap: float = 0.5 # min internal gap (s) that qualifies as a split
point
141         # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
142         # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
143         # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
144         # Only cuts (recall can't drop). OFF by default until measured/promoted.
145         window_hard_cap: bool = False
146         # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
147         # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
148         # [?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
149         # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
150         # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
151         # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, [?end| 4.96?3.31s (n=13).
The W1 cap
152         # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
153         window_inactivity_end: float = 1.5
154         inactivity_rally_thresh: float = 0.20
155         inactivity_min_density: float = 0.30
156         # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
157         # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group

```

```

STILL longer
158         # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
159         # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
160         # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
161         # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
162         # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
163         # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
164         window_blob_fallback: bool = False
165         # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
166         # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
167         # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
168         # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
169         window_blob_factor: float = 4.0
170         log_yolo_candidates: bool = True
171         store_yolo_candidates: bool = True
172         ai_handoff_padding: float = 2.0
173         ai_max_workers: int = 5
174         # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
175         # chunks of a high-bitrate (4K) source blow past the provider upload cap
176         # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
177         # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
178         # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
179         # Matches gemini_refine's --clip-scale-width default.
180         ai_chunk_scale_width: int = 1280
181         # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
182         # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
183         # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
184         # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
185         # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
186         skip_ai_handoff: bool = False
187         # Optional debug knob: trim every video to the first N seconds before processing.
188         debug_duration: Optional[float] = None
189
190         tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
191         wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
192         fusion: FusionConfig = Field(default_factory=FusionConfig)
193
194         @field_validator("default_segmenter")
195         @classmethod
196         def segmenter_must_be_registered(cls, v: str) -> str:
197             # Registry check happens at runtime in main/segmenter selection.
198             # Here we just allow any string for forward compatibility.
199             return v
200
201         @model_validator(mode="after")
202         def _chunk_step_must_be_positive(self) -> "IndexingConfig":
203             # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
204             # step makes `while t < duration: t += step` loop forever, growing memory before
any
205             # GPU work. Reject the bad config at load time instead.
206             if self.chunk_overlap >= self.chunk_duration:

```

```

207         raise ValueError(
208             f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
209         "
210             f"({self.chunk_duration}); otherwise chunking never advances."
211         )
212         return self
213
214     class OutputConfig(BaseModel):
215         default_highlights_name: str = "highlights.mp4"
216         db_name: str = "sports_indexer.db"
217         # Directory where the DB, highlight reels, and processed clips are written.
218         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
219         output_dir: str = "output"
220
221     class WatermarkConfig(BaseModel):
222         """Branding overlay burned into each highlight clip during the per-clip re-encode
223         (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
224         (under `stitching.watermark` in config.json) to turn it off. The text is fully
225         configurable. The concat stage stays stream-copy (-c copy)."""
226         enabled: bool = True
227         text: str = "Generated by Khelsutra.guru"
228         logo_path: str = "" # optional transparent PNG overlay; "" = no logo
229         font_path: str = "" # optional .ttf; "" = use the fontconfig `font` family
230
231         below
232         font: str = "Sans" # fontconfig family used when font_path is empty
233         font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5) # text height as a
234         fraction of frame height
235         opacity: float = Field(default=0.85, ge=0.0, le=1.0)
236         box: bool = True # faint backing box behind the text for legibility
237         margin: int = Field(default=24, ge=0) # px inset from the chosen corner
238         position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
239         right"
240
241     class StitchingConfig(BaseModel):
242         begin_padding: float = 0.5
243         end_padding: float = 0.5
244         use_cache: bool = False
245         min_shot_count: int = 1
246         # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
247         duration
248         # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
249         # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
250         the
251         # fully-local no-AI path, whereas duration is derived from the rally's own
252         start/end. The
253         # local detector over-fires and its false positives are mostly SHORT (motion blips,
254         # background-court fragments), so this keeps the reel to the eye-catching long
255         rallies.
256         # OFF by default ? the detector output is unchanged, only which rallies reach the
257         reel.
258         min_rally_duration: float = Field(default=0.0, ge=0.0)
259         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
260
261     class LoggingConfig(BaseModel):
262         """Logging knobs for the run entrypoints (see backend/utls/logging_setup.py)."""
263         # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
264         # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
265         # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
266         # them to WARNING; set true to restore full chatter for request-flow debugging.
267         verbose_http: bool = False

```

```

263
264 class SecurityConfig(BaseModel):
265     # When set, /api/process video_path must resolve under this directory (hosted/tenant
266     # mode). Default None preserves the single-user local 'any local file' workflow.
267     allowed_video_root: Optional[str] = None
268
269     # --- Chunked upload (B2, PR-2) ---
270     # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
271     # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
272     # contain upload_dir or /api/process will reject the uploaded paths.
273     upload_dir: str = "output/uploads"
274     # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
275     # free-tier edge proxies cap request bodies there (HOSTING_PLAN $2).
276     max_upload_mb: int = 4096
277     max_part_mb: int = 95
278     allowed_upload_extensions: List[str] = ["mp4", "mov"]
279
280
281 class AppConfig(BaseModel):
282     """Root configuration object.
283
284     All runtime configuration should be accessed via an instance of this class
285     (or the loader) rather than raw dict lookups.
286     """
287
288     sport: Sport = "badminton"
289     ai_provider: str = "gemini"
290     # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
291     # the alias never deprecates underneath us. Eval reports record the run date
292     ai_model: str = "gemini-flash-latest"
293
294     validation: ValidationConfig = Field(default_factory=ValidationConfig)
295     indexing: IndexingConfig = Field(default_factory=IndexingConfig)
296     output: OutputConfig = Field(default_factory=OutputConfig)
297     stitching: StitchingConfig = Field(default_factory=StitchingConfig)
298     security: SecurityConfig = Field(default_factory=SecurityConfig)
299     logging: LoggingConfig = Field(default_factory=LoggingConfig)
300
301     # Allow extra keys during transition so existing config.json files do not break.
302     model_config = {
303         "extra": "allow",
304         "validate_assignment": True,
305     }
306
307     # Temporary migration shim for legacy dict-style access (config.get("section",
308     {}).get(...)).
309     # Will be removed once all call sites (validators, segmenters, etc.) move to typed
310     # attribute access / the typed config. A nested *section* is returned as a plain
311     # via model_dump, so legacy chained access keeps working ? returning the Pydantic
312     # section model instead breaks it (a model has no .get), which bit every validator
313     # real runs. Scalar leaves are returned as-is.
314     def get(self, key: str, default: Any = None) -> Any:
315         _MISSING = object()
316         val: Any = _MISSING
317         if key in type(self).model_fields or hasattr(self, key):
318             val = getattr(self, key, _MISSING)
319         if val is _MISSING:
320             # Legacy convenience: find a leaf key inside any nested section. Sections
321             # discovered dynamically from the model fields, so a future top-level
322             section

```

```

321         # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
322         for fname in type(self).model_fields:
323             section = getattr(self, fname, None)
324             if isinstance(section, BaseModel) and hasattr(section, key):
325                 val = getattr(section, key)
326                 break
327         if val is _MISSING:
328             return default
329         if isinstance(val, BaseModel):
330             return val.model_dump(mode="json")
331         return val
332
333     def __getitem__(self, key: str) -> Any:
334         val = self.get(key)
335         if val is None and key not in type(self).model_fields:
336             # Re-raise like a real dict for unknown top-level
337             raise KeyError(key)
338         return val
339
340
341     # Convenience type alias for callers that want to annotate
342     Config = AppConfig
343

```

7. assistant (2026-06-19T15:46:43.929Z)

Now let me read the five target docs.

8. user (2026-06-19T15:46:44.861Z)

```

1     # Multi-signal rally fusion ? add a human (person) cue alongside the shuttle
2
3     **Status:** **P1 DONE** (2026-06-13) ? the person cue is extracted to
4     `backend/pipeline/detectors/person_detector.py` (`PersonDetector`), consumed by
`yolo_hybrid` with no
5     behavior change. **P2?P4 remain owner-gated.** Phases below are sequenced but each is a
separate PR.
6     **Owner decisions captured** in [S9](#9-open-decisions-for-the-owner).
7
8     > **Companion docs:** the 2-layer mental model is
[HOW_RALLY_DETECTION_WORKS.md](HOW_RALLY_DETECTION_WORKS.md);
9     > the modular-fusion architecture this builds on is **ADR-007**
([archives/decisions/DECISIONS.md](archives/decisions/DECISIONS.md));
10    > the permissive-dependency rule is **ADR-002 / ADR-005** ; the audio cue (the #1 next
cue, already
11    > scaffolded) is [archives/research/AUDIO_RALLY_DETECTION_PLAN.md](archives/research/AUD
IO_RALLY_DETECTION_PLAN.md).
12    > The vendor labels that gate the learned path come from
[GOLDEN_SET_VENDORING.md](GOLDEN_SET_VENDORING.md)
13    > and the labeling spec in [ROORA_LABELING_GUIDELINES.md](ROORA_LABELING_GUIDELINES.md).
14    > **How each cue here is added/tested without production regression** ? A/B, shadow
eval, and the
15    > promote-only-on-lift discipline ? is [EXPERIMENT_HARNESS.md](EXPERIMENT_HARNESS.md)
(every P1?P4 cue
16    > below lands flag-gated, default OFF, promoted only on measured LOVO lift).
17
18    ---
19
20    ## 1. Context ? why this, why now
21
22    Rally detection today runs **one cue at a time**. Each segmenter independently turns a
single signal
23    into rally windows:
24

```

```

25     | Segmenter | Signal | File |
26     |---|---|---|
27     | `wasb_hybrid` / `tracknet_hybrid` | shuttle velocity (frozen WASB/TrackNet trajectory)
| `backend/pipeline/detectors/tracknet_runner.py` |
28     | `yolo_hybrid` (the Pydantic `default_segmenter` fallback; `config.json` overrides it
to `gemini`) | person velocity (torchvision + ByteTrack) |
`backend/pipeline/segmenters/yolo_hybrid.py` |
29     | `motion` | raw pixel motion | `backend/pipeline/segmenters/motion.py` |
30     | `gemini` | a cloud VLM watching the whole clip | `backend/providers/gemini.py` |
31
32     **There is no fusion.** The owner's directive: make the detector **use the shuttle *and*
the humans
33     together** ? and be ready to fold in more signals (audio, later pose) ? delivered
34     **MIT / Apache-2.0 / BSD-clean** so we can ramp commercially without a licensing
rethink.
35
36     This is **not greenfield** ? it operationalizes work the repo already blessed:
37
38     - **`HOW_RALLY_DETECTION_WORKS.md` open question:** "Fusion: combine shuttle-motion
(WASB) with
39     player-motion for static cams."
40     - **ADR-007** already designed a **modular "rally layer"** that fuses cues *outside* the
frozen WASB
41     model, with **audio as the #1 next cue and person/pose parked as #2**. This plan picks
up the parked
42     person cue and generalizes the fusion seam so future cues are "register a producer,"
not "re-touch
43     the segmenter."
44     - **`backend/pipeline/detectors/rally_gate.py` already exists** ? the net-crossing gate,
explicitly
45     labelled in its own docstring as "Gate A in the over-fire fix spectrum (**B = player-
stance state
46     machine, future**)." The person cue in this plan **is that named-but-unbuilt Gate
B.**
47
48     ### The owner's "held-shuttle" false positive ? what's actually going on
49
50     The owner observed a "rally" that was just a player **holding / flicking the shuttle in
hand** between
51     points. This is the *exact* failure `rally_gate.py` was written to kill:
52
53     > "WASB over-detects: when a player flicks/tosses the shuttle back to the opponent for
service after
54     > losing a point, that single trajectory looks like a rally. The discriminator is
structural: a real
55     > rally has the shuttle crossing the net MULTIPLE times; a single service feed crosses
~once."
56
57     **The structural fix already exists and is validated offline ? but it is NOT wired into
the production
58     runtime path.** `rally_gate.apply_gate(..., min_crossings=2)` is called only from the
**eval drivers**
59     (`eval_partial_labels.py`, `export_wasb_segments.py`, `calibrate_local.py`;
`distill_local.py` uses
60     `rally_gate.gate_windows` to compute the `net_crossings` feature, not `apply_gate`); the
live segmenters
61     historically didn't apply it. P2 closed this: `FusionConfig.min_crossings` now exists
(default `0` =
62     OFF), so the served Gate A is a config flag ? consistent with §10 P2 below, marked DONE.
So the
63     single cheapest, highest-confidence win in this whole plan is **P2's first step: fold
the existing
64     Gate A into production.** Everything else (the person cue / Gate B, the learned fusion)
is robustness
65     on top of that.

```

```

66
67     ### Why humans help ? and the honest risk
68
69     The beachhead videos are static tripod (Bangalore academies,
[PMF_MARKET_RESEARCH.md](PMF_MARKET_RESEARCH.md)),
70     so player presence/motion is a usable cue. ADR-001's warning was specifically about
broadcast pan
71     cameras, where player-motion fooled the old heuristic ? that failure mode does not
apply to a fixed
72     court cam.* But we keep the shuttle as the camera-invariant primary so the general
bet stays robust
73     on other camera types. Humans then add:
74
75     - Precision ? reject shuttle-on-floor / warm-up knock-ups / held-shuttle feeds when
the court
76     isn't occupied by 2 players in a play posture.
77     - Recall (via the learned classifier) ? features that help bridge shuttle
occlusion/blur gaps.
78
79     ---
80
81     ## 2. Core architecture ? one rally-fusion layer, cues are swappable producers
82
83     Formalize ADR-007's diagram. Every cue is a black box that emits a time-aligned
signal* (per-frame
84     activity and/or per-window features); fusion happens in our layer, never inside a
frozen model.**
85
86     ```
87     frames ?? WASB (frozen, MIT)      ?? shuttle trajectory ???
88     frames ?? person detector (ours)  ?? tracks + boxes ??????
89     audio ?? hit detector (ours)      ?? hit events ???????????? RALLY LAYER (ours) ??
rallies
90     [frames ?? pose/stance (ours; parked #3) ?? stance ???????? feature + decision fusion
91     ```
92
93     - Shuttle = primary, camera-invariant. Windows still seed from
94     `TrackNetRunner.trajectory_to_action_windows()` (`tracknet_runner.py:234`). Other cues
enrich and
95     adjudicate* shuttle-seeded windows; they do not replace the shuttle seed. Person
runs only over
96     shuttle-seeded candidate windows + padding, not the whole video ? which bounds the
added cost.
97     - A small `RallyCue` contract. Each producer yields `(frame_idx ? features)` + per-
window stats.
98     Adding a cue = register a producer; the existing `SegmenterRegistry`
(`segmenters/base.py:35`) is the
99     template for the pattern. Keep the single-cue segmenters registered as fallbacks.
100
101     ---
102
103     ## 3. The two fusion modes (both ? per the owner decision)
104
105     ### 3.1 Decision-level gate (safe baseline, ships first, no labels needed)
106
107     ```
108     rally_active = shuttle_motion ? (2 persons in court zone ? recent person track)
109     ```
110
111     The `? recent-track` clause tolerates 1/2 frame person-detector misses so the AND-gate
can't cause a
112     recall cliff. This composes with the existing net-crossing Gate A:
113
114     ```
115     keep window ? net_crossings ? 2                (Gate A ? EXISTS in rally_gate.py, eval-

```

```

only today)
116             ? (?? in-court players ? recent track)    (Gate B ? the person cue, NEW)
117     ...
118
119     Gate A kills the service-feed / held-shuttle case structurally; Gate B adds court-
occupancy evidence
120     for the cases a single trajectory can still fool (e.g. a knock-up that does cross the
net). No training
121     data required ? this is the label-free fusion that ships before any golden labels exist.
122
123     ### 3.2 Feature-level (primary, the learned path)
124
125     The cues emit a **small set of aggregated, scale/translation-invariant per-window
features ? NOT raw
126     coordinates.** Raw pixel coordinates would memorize *this* court/camera and fail to
generalize off a
127     still-small labeled set; counts / ratios / zone-membership / two-sidedness transfer.
128
129     **What already feeds the classifier** (`distill_local.py:40`, `FEATURE_NAMES`):
130     `duration, peak_velocity, mean_velocity, active_ratio, net_crossings` ? note
**`net_crossings` is
131     already a feature** (computed via `rally_gate.gate_windows`).
132
133     **Person features to add (~4, deliberately few):**
134
135     | Feature | Meaning | Rally signal |
136     |---|---|---|
137     | `players_in_court` | median count of persistent in-court tracks | ? 2 (singles) / 4
(doubles); 0?1 = dead time |
138     | `two_sided_motion` | are moving players on **both** net-halves? | rally = two-sided;
one person feeding = one-sided |
139     | `mean_player_motion` | aggregate normalized player velocity | rally = sustained;
resting/standing = low |
140     | `in_court_ratio` | fraction of frames with ?2 in-court players | track stability vs
flicker |
141     | `shuttle_player_colocation` | fraction of frames the shuttle sits inside/adjacent to a
person box (hand region) | **held = high; in-flight = mostly free space** ? the direct held-
shuttle discriminator |
142
143     These join the shuttle features in the candidate `stats` JSON
144     (`database.py::add_candidate_segment`) and the feature vector of the **existing
distilled classifier ?
145     the tiny numpy-only L2 logistic regression in `backend/eval/classifier.py` (ADR-004),
NOT a new net.**
146     It scores each candidate window `P(real rally)` and replaces the Gemini call at runtime.
147
148     **What the model learns:** the **weights** on those few features ? e.g. down-weight a
shuttle-motion
149     window with 0?1 in-court players / one-sided motion / high shuttle-colocation (warm-up,
floor shuttle,
150     held-shuttle feed); up-weight ?2 players, two-sided, sustained motion, shuttle in free
flight.
151
152     **Small-N discipline (non-negotiable):**
153     - Keep to **~4 person features** ? more features on a still-small labeled set overfit;
that's exactly
154     why the model stays a logistic regression, not a net.
155     - Use the **leave-one-video-out (LOVO)** protocol that `distill_local.py` **already
implements**: train
156     on the other videos' candidates, predict the held-out *video*, score vs its labels ?
never a
157     held-out *window* from the same video (windows in one video are correlated; that
flatters the score).
158     - Training labels today are **Gemini silver labels** (distilled offline, ADR-004/007 ?
Gemini is a

```

```

159     teacher, never a runtime dependency, never a training-data *source* for owned
weights); **golden
160     labels are the honest measuring stick**, and as the Roora golden corpus grows they can
also become
161     training labels. Until the labeled set is larger, treat the learned weights as
162     **calibration / direction-check, not "a fully general model"*** ? the decision-level
gate (§3.1) is
163     the shippable, label-free fusion in the meantime.
164
165     > *(Gated on golden labels existing ? the same blocker ADR-007 flags for audio. See
166     > [GOLDEN_SET_VENDORING.md](GOLDEN_SET_VENDORING.md).)*
167
168     ---
169
170     ## 4. Shuttle-state: in-flight vs in-hand (the misclassification, mapped to real code)
171
172     The "shuttle moved" seed conflates a shuttle **in flight** with a shuttle **carried in
hand**. The
173     discriminators, and where each lives:
174
175     | Signal | In-flight | Held / static | Status |
176     |---|---|---|---|
177     | `net_crossings` | ? 1 (usually several) | ~0 (held) / ~1 (single feed) | **EXISTS**
(`rally_gate.py`); eval-only ? wire to prod |
178     | `shuttle_player_colocation` | low (free space) | high (near a hand) | **NEW** (needs
the person cue) |
179     | `direction_reversals` | several (racket contacts) | ~0 | derivable from trajectory;
partial overlap with crossings |
180     | `peak shuttle speed` | high | walking speed | derivable from existing velocity |
181
182     So the in-flight-vs-held fix is **mostly Gate A (already built) + the new colocation
feature from the
183     person cue**. **No new label column is needed** ? the golden labels already start a
rally at the
184     *serve-contact frame* and treat all pre-serve / held-shuttle time as dead time (see
185     [ROORA_LABELING_GUIDELINES.md](ROORA_LABELING_GUIDELINES.md)), so held-shuttle moments
are negatives
186     by construction and are the ground truth that proves the detector wrong.
187
188     ---
189
190     ## 5. Adversarial mitigations (integrating safely)
191
192     - **Court-region mask** ? polygon around the court; drop persons outside it (spectators,
coaches,
193     adjacent courts). Config-supplied per camera; ~1 ms. Source: a manual per-camera
polygon first;
194     the **4 court corners + net-line** that Roora labels once per video (see the labeling
doc) build
195     this mask **and** the net split that `two_sided_motion` / `net_crossings` need.
196     - **Player-count constraint** ? expect **2 (singles) / 4 (doubles)** persistent in-court
tracks;
197     suppress transient/extra detections. The `singles_or_doubles` manifest flag sets the
expectation.
198     - **Temporal association** ? reuse `supervision` **ByteTrack** (already in
`yolo_hybrid`); reject
199     singleton detections; smooth across frames.
200     - **Multi-court rejection** ? proximity to the primary court region (homography
optional, later).
201     - **Domain shift (COCO frontal ? side/overhead court)** ? start **box-only** (robust);
keep the
202     detector swappable. Fine-tuning a person detector is **parked** ? we have no in-house
person labels,
203     and the **paid-LLM/training guardrail + minors-exclusion** apply to any future person-
training data.

```

```

204 - **Fusion-degradation guard** ? shuttle stays primary; person is gate + feature, never
the sole
205 trigger; the single-cue segmenters remain registered fallbacks.
206
207 ---
208
209 ## 6. Box vs pose
210
211 **Box-first.** Occupancy + count + two-sided motion + shuttle-colocation is enough for
both the gate
212 and the features, and box detection is the cheap, robust win. **2D pose is parked behind
the same
213 `RallyCue` interface** ? pose (MediaPipe Pose / RTMPose, both Apache-2.0) buys serve-
ready stance
214 gating and shot cues later, at the cost of a second model. This matches ADR-007 parking
pose as the
215 #2/#3 cue.
216
217 ---
218
219 ## 7. Performance
220
221 The person detector is the **existing in-process torchvision model** run at **frame-skip
375**
222 (`yolo_frame_skip` default 3) with track interpolation between sampled frames. The
shuttle keeps
223 running in its separate WSL process ? the two are **separate processes**, so a "shared
backbone" isn't
224 free; budget realistically as the existing `yolo_hybrid` cost minus skip savings. Person
runs **only
225 over shuttle-seeded windows + `ai_handoff_padding`, not the whole video, which bounds
the cost.
226
227 ---
228
229 ## 8. Licensing (the guardrail table ? all green)
230
231 Every dependency stays **MIT / Apache-2.0 / BSD** (ADR-002 / ADR-005; `ultralitics` was
dropped for
232 AGPL-3.0).
233
234 | Cue / model | Code | Weights / data | Verdict |
235 |---|---|---|---|
236 | Shuttle: WASB-SBDT, TrackNetV3 | MIT | frozen badminton weights | ? (ADR-001/002) |
237 | Person (default): torchvision Faster R-CNN / RetinaNet / FCOS | BSD/MIT | COCO-
pretrained | ? already in repo (`_DETECTOR_FACTORIES`) |
238 | Person (escalation): **YOLOX**, **RT-DETR / RT-DETRv2** | Apache-2.0 | COCO /
Objects365 | ? |
239 | Pose (parked): **MediaPipe Pose**, **RTMPose / MMPose** | Apache-2.0 | COCO-structure
| ? |
240 | Tracking: `supervision` ByteTrack | MIT | n/a | ? already in repo |
241 | **Banned** | **Ultralytics YOLOv5/8/11 (AGPL-3.0)** . **MonoTrack (Adobe, non-comm)**
. **YOLO-NAS (Deci, non-comm)** | ? | ? |
242
243 **COCO caveat to record:** COCO *annotations* are **CC BY 4.0** (commercial OK *with
attribution*);
244 COCO *images* are Flickr-ToS (per-image). Pretrained-**weight** use is clean; record the
attribution.
245
246 ---
247
248 ## 9. Open decisions (for the owner)
249
250 1. **Court-region mask source** ? manual per-camera polygon (simple, ship first) vs auto
court

```

```

251     homography/line detection (later). **Recommend manual first**, fed by Roora's court-
corner labels.
252     2. **Person detector default** ? keep torchvision (zero new deps, BSD/MIT) vs adopt
YOLOX/RT-DETR
253     (Apache, faster) now. **Recommend torchvision first**, escalate only if eval shows a
speed/accuracy gap.
254     3. **Pose now or parked** ? **recommend parked** behind the interface (box-first).
255     4. **Golden labels** ? feature-level fusion (P3) is **blocked on golden rally labels**;
the source is
256     the VLC `rally-annotator` + the Roora vendor pilot
([GOLDEN_SET_VENDORING.md](GOLDEN_SET_VENDORING.md)).
257
258     ---
259
260     ## 10. Phased execution (owner-gated; ONE PR per slice, off `master`)
261
262     - **P0 ? this design doc** +
[ROORA_LABELING_GUIDELINES.md](ROORA_LABELING_GUIDELINES.md), indexed in
263     `docs/README.md`, cross-linked from `HOW_RALLY_DETECTION_WORKS.md` and ADR-007. *(this
deliverable;
264     docs-only, no code.)*
265     - **P1 ? extract the shared person cue:** ? **DONE (2026-06-13).** Lifted the
torchvision detector +
266     ByteTrack + velocity out of `yolo_hybrid` into
`backend/pipeline/detectors/person_detector.py`
267     (`PersonDetector`); `yolo_hybrid` now orchestrates only and delegates to `self.person`
(no behavior
268     change ? regression-safe, proven by the unchanged process_video output tests). New
269     `tests/test_person_detector.py`; suite +1 green; person_detector is type-clean (left
mypy quarantine
270     for the moved code). The fusion segmenter (P2) reuses this same producer over shuttle-
seeded windows.
271     - **P2 ? the FusionSegmenter (Gate A + Gate B + person-feature persistence):** ? **DONE
(2026-06-14).**
272     Shipped `backend/pipeline/segmenters/fusion_hybrid.py::FusionSegmenter` (registry
`fusion_hybrid`,
273     **default-OFF** ? `default_segmenter` unchanged), subclassing the trajectory engine
via two identity
274     hooks (`_enrich_candidate_stats`, `_select_for_handoff` ? wasb/tracknet stay byte-
identical, adversarially
275     verified). It seeds windows from the shuttle substrate, **always persists
`net_crossings`** (Gate A signal),
276     and ? when `indexing.fusion.cue_flags.person` is on ? runs the person cue over each
window's frames
277     (`PersonDetector.detections_per_frame`) and persists the ~5 person features
(`fusion_features.py`,
278     pure + unit-tested). Served gating is optional + default-OFF: `fusion.min_crossings`
(Gate A) /
279     `fusion.min_players` (Gate B) skip sub-threshold windows from the handoff while
**persisting the full
280     superset** (offline substrate intact). Court mask = optional `fusion.court_polygon`
(None ? count-only).
281     Lift to be measured via the harness; held-shuttle-drop + person-feature values are
GPU/owner-side.
282     - **P3 ? feature-level (learned) fusion:** add the person features (incl.
`shuttle_player_colocation`)
283     to `distill_local` / `classifier`; recalibrate IoU on the golden set; report **LOVO
lift on held-out
284     videos**. *(Gated on golden labels.)*
285     - **P4 ? additional cues:** wire **audio** (ADR-007's #1, already scaffolded in
286     `backend/pipeline/audio/hit_detector.py`) and later **pose**, through the same
`RallyCue` interface.
287     Each is add-a-producer, not a refactor.
288
289     ---

```

```

290
291     ## 11. Verification
292
293     - **Offline, runnable in this dev container (no GPU/torch/cv2):** unit tests for the
fusion gate logic
294         and `RallyCue` aggregation (court-mask filter, player-count constraint, `? recent-
track` tolerance)
295         and the shuttle-state signals (net-crossing count ? extends the existing `rally_gate`
tests ? and
296         person-colocation) with **synthetic trajectories + person boxes**. A key-license
assertion test
297         (no `ultralytics` import; detector factories stay torchvision/Apache).
298     - **Machine-side (owner, GPU/WSL ? the full pipeline can't run in this container):** run
the
299         `FusionSegmenter` on a real static-tripod clip; confirm **held-shuttle / warm-up /
spectator false
300         positives drop** vs shuttle-only (this should reproduce the owner's observed bug and
show it fixed ?
301         start by confirming Gate-A-in-production alone fixes it); **LOVO IoU lift on the
golden set** for the
302         learned path.
303     - **Docs (P0):** links resolve; both docs indexed in `docs/README.md`; this doc cross-
linked from
304         `HOW_RALLY_DETECTION_WORKS.md` (the fusion open-question) and ADR-007.
305

```

9. user (2026-06-19T15:46:45.392Z)

```

1     # Rally Detection ? Closing the Accuracy Gap (golden-eval-driven, per-video)
2
3     > **Status:** `DRAFT` ? implementation **owner-gated** . **Owner:** Avi . **Created:**
2026-06-18 . **Last updated:** 2026-06-18
4     > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
5     > **Tracking anchors:** $7 progress tracker is the source of truth; pointer in memory
`current-state`; add to `docs/README.md` + `docs/NEXT_STEPS.md` when work starts.
6     > **Relation to existing docs:** **peer-of**
[ `RALLY_QUALITY_RESEARCH.md` ] (RALLY_QUALITY_RESEARCH.md) (that is the research + eval-framework
doc; this is the *execution tracker* for the remaining accuracy gap). **Builds on**
[ `MULTI_SIGNAL_FUSION_PLAN.md` ] (MULTI_SIGNAL_FUSION_PLAN.md) (signal mechanics),
[ `QUALITY_ITERATIONS.md` ] (QUALITY_ITERATIONS.md) (the R-log / empirical backbone),
[ `GOLDEN_VIDEOS.md` ] (GOLDEN_VIDEOS.md) (the corpus). Does **not** duplicate them ? it points.
7     > **Honesty note:** each claim tagged `[verified]` (checked vs code/measured) /
`[design]` (proposed). The headline numbers below are real measurements, but `n` is small ? read
$6.
8
9     ---
10
11     ## 0. TL;DR
12     We are using the **growing golden corpus** (human rally labels) to find and close the
rally-detection accuracy gap **per video, before launch** ? and the first ground-truth clip
already reshaped the picture. The boundary work (R2/R3/R4) **holds** ? when local fires on a
real rally, its edges are tight ( $|?| < 1$  s). **The remaining gap is DETECTION, not boundaries:**
(G1) **recall** ? both local and Gemini miss ~half the rallies on hard clips; (G2) **precision**
? local over-fires (false positives), especially on continuous drilling; (G3) **rally-END over-
extension** ? when a player *quickly knocks the shuttle back after a point*, shuttle-velocity
alone can't tell "rally" from "casual reset," so the window runs long / merges. n=2 sharpens the
priority: **LOCAL = high recall / low precision, GEMINI = high precision / low recall**
(opposite, complementary ? on the normal clip local's recall *beats* Gemini's). So **the #1
lever is precision** (cut local's false positives via `rally_gate` Gate-A + colocation; the
player-state rally-END rule feeds this), with **signal fusion** (local's recall + Gemini-style
precision) as the durable win. **Caveat:** two golden clips so far (n=13 corpus,
GX010141+GX010137) ? directional, not conclusive.
13
14     ## 1. Problem & goal

```

15 The detector is good enough to ship a highlights product, but the golden eval shows clip-dependent accuracy that we want to ****close consistently on every video, not just on average**** ? catching the failure modes in the quality loop instead of after launch.

16

17 - ****Concrete target:**** on every corpus video, local is ****at-par-or-better than Gemini vs the golden labels****, with ****no per-video regression**** on either eval set (the dual-eval-set discipline, `[[eval-dual-set-regression]]`).

18 - ****"Good" =**** the three gaps below closed enough that a milestone flip (default-ON) clears the over-seg guard AND improves per-video F1 broadly, recorded in a Launch Report (`[[launch-report-convention]]`).

19 - ****Read first to get current:****
`[`RALLY_QUALITY_RESEARCH.md`](RALLY_QUALITY_RESEARCH.md) $1?2 (the gap + diagnosis),`
`[`MULTI_SIGNAL_FUSION_PLAN.md`](MULTI_SIGNAL_FUSION_PLAN.md) $4 (in-flight vs in-hand),`
`[`HOW_RALLY_DETECTION_WORKS.md`](HOW_RALLY_DETECTION_WORKS.md) (the 2-layer model).`

20

21 **## 2. Decisions locked**

22 | # | Decision | Source / date | Implication |

23 |---|-----|-----|-----|

24 | D1 | The gap is ****DETECTION (recall + precision + rally-END), not boundary precision**** | golden GX010141, 2026-06-18 ``[verified]`` | Don't build a generic boundary head; spend on rally-state signals. |

25 | D2 | Measure ****per-video AND aggregate; no regression on ANY video**** | `[[eval-dual-set-regression]]` / `[[decision-rigor-norm]]` | A win must broadly help, not trade one clip for another. |

26 | D3 | ****Golden labels = ground truth; Gemini = reference, not truth**** | n=12 corpus | Gemini under-detects too ? agreement ? accuracy. Report both, gate on golden. |

27 | D4 | New signals land ****default-OFF + flag-gated****, promoted only on measured lift | `[`EXPERIMENT_HARNESS.md`](EXPERIMENT_HARNESS.md)` / `[[weak-signals-retained]]` | Zero-regression rollout; retained even on a small-n null. |

28 | D5 | ****Rally-END needs a PLAYER-state signal**** ? shuttle velocity can't separate a competitive rally from a casual post-point return | owner insight + R4 limitation, 2026-06-18 ``[design]`` | Add person kinematics / shuttle-in-hand as the rally-end discriminator. |

29

30 **## 3. Foundation ? the measured gaps (golden ground truth)**

31 Ground-truth-scored Boxhill clips (more land as the owner labels them). Milestone windowing = cap6 + R4 (`[`QUALITY_ITERATIONS.md`](QUALITY_ITERATIONS.md)` R3/R4). Local & Gemini vs truth, n=2 so far: ``[verified]``

32

33 | clip (golden) | system | found | tIoU@0.5 P/R/F1 | tolF1 | hits | false+ | missed |

34 | \?start\ | \?end\ |

35 |---|---|---|---|---|---|---|---|

35 | ****GX010141**** (29, hard/drilling) | LOCAL | 22 | 0.59/0.45/****0.51**** | 0.43 | 13 | 11 |

16 | 0.78 | 0.68 |

36 | | GEMINI | 16 | 0.94/0.52/****0.67**** | 0.53 | 15 | 4 | 14 | 0.31 | 0.46 |

37 | ****GX010137**** (34, normal match) | LOCAL | 39 | 0.77/****0.88****/****0.82**** | 0.68 | 30 | 14 |

9 | 0.39 | 0.44 |

38 | | GEMINI | 27 | 1.00/0.79/****0.89**** | 0.89 | 27 | 0 | 7 | 0.50 | 0.52 |

39

40 - ****Corpus aggregate (n=13, milestone):**** tIoU F1@0.5 ****0.474**** [0.359, 0.579], F1@0.25 0.633; R2 tolF1 0.359. GX010137 (0.822) is the corpus's ****best**** clip; GX010141 (0.510) is above the mean. ``[verified]``

41 - ****Opposite failure modes ? the central finding (n=2).**** ****LOCAL = high recall, low precision; GEMINI = high precision, low recall.**** On the normal clip GX010137, local's ****recall 0.88 BEATS Gemini's 0.79**** (local finds ****more* real rallies**) but local fires 14 false positives; Gemini fires ****zero**** false positives but misses 7 rallies. They are ****complementary**** (on GX010141 they overlap on only ~3 rallies yet each hits ~13?15 of the golden set). ? ****fusion (local's recall + Gemini-style precision filtering) is the clear win.****

42 - ****The gaps, re-prioritized by the data:****

43 - ****G2 ? precision is LOCAL's #1 gap**** (over-fires on both clips: 11/22 and 14/39 false positives). This is the cheapest + highest-leverage fix (Gate-A / colocation). Filter local's FPs on GX010137 and it lands ~0.88+ ? at-par with Gemini, since local already out-recalls it.

44 - ****G1 ? recall**** is clip-dependent: a local ****strength**** on normal play (? Gemini), but on hard continuous-drilling clips ****both**** miss ~half (GX010141 recall ~45?52%). The recall lever matters mainly for the hard clips.

45 - ****G3 ? rally-END over-extension**** (quick-return-after-point): shuttle keeps moving ? window runs long / merges; R4 (shuttle-only) can't catch it. Contributes to G2's false positives/merges.

46 - ****Local's gap to Gemini SHRINKS on normal play:**** hard clip ??0.16 (0.51 vs 0.67) ? normal clip ??0.07 (0.82 vs 0.89), and local *out-recalls* Gemini there. The deficit is precision, not detection.

47 - ****What is NOT the gap:**** boundary precision. When local hits a rally its edges are tight (|?start|/?end| ? ~0.8 s, on GX010137 even tighter than Gemini). The R2/R3/R4 windowing is doing its job. `[verified]`

48 - ****Long-rally lens refinement (P7, n=15, `[verified]`):**** filtering the eval to >5s rallies (the eye-catching highlights) shows local's single-court over-fire is ****short-FP-dominated**** (seg_ratio 1.43?1.04 ? duration-filterable), but ****multicourt still over-fires 1.58x on LONG rallies**** (adjacent-court games are genuine long rallies). ? the multicourt precision gap needs ****court-localization (Gate-B / court mask, Lever C)****, NOT a duration filter; and the strict tolerance precision is gated by ****over-merge contamination**** of the long-pred set, not just short FPs. See QUALITY_ITERATIONS.md "P7".

49 - ? ****Data caveat:**** GX010141 was labeled before ~3 rally-annotator-tool bugs were found + fixed (GX010137 is post-fix). GX010141's numbers may shift on a re-export ? re-verify before drawing hard conclusions from it.

50

51 **## 4. Design / architecture ? the levers (REUSE MAP)**

52 The signal mechanics live in MULTI_SIGNAL_FUSION_PLAN.md; this is which lever attacks which gap.

53

54 - ****Lever A ? player-state rally-END rule**** (targets ****G3 + G2****) `[design]`. Generalize R4 from shuttle-only to ****shuttle OR bilateral player motion****: end the rally at the last frame with competitive evidence from *either*, and ****hard-terminate on a "shuttle picked up" event****. Two cheap detectors, no new model required:

55 - ****Player kinematics* ? person velocity drops / players go stationary after a point** (the casual returner taps it while the other walks). ****Reuse:**** the person cue producer (torchvision + ByteTrack) in `yolo\_hybrid` (fusion P1, `[verified]` exists).

56 - ****Shuttle-in-hand / out-of-play* ? `shuttle\_player\_colocation`** (shuttle box ? person box = held) `[design]` from the fusion plan §4, ****or**** the no-new-model proxy: ****sustained WASB `Visibility=0`**** after the last fast stroke (held shuttle goes dark). Use it as a ****termination anchor + reverse-timeline search**** to the true end (last net-crossing / last fast-shuttle / last bilateral motion) ? the owner's idea, refined: the hold event is *late*, so anchor-then-backtrack, don't use its timestamp as the end.

57 - ****Lever B ? recall**** (targets ****G1****) `[design]`. Both systems miss real rallies ? detector sensitivity: proposal-windowing / velocity-threshold tuning, and the ****complementary-fusion**** angle below. Measure recall floor per video; the highest-leverage labels are the ****missed* rallies (active learning, RALLY_QUALITY_RESEARCH §4.3).**

58 - ****Lever C ? signal/ensemble fusion**** (targets ****G1****) `[design]`. Local & Gemini (and shuttle vs person cues) hit ****different* rallies ? a union/ensemble lifts recall.** Already-built scaffolding: `rally\_gate.py` ****Gate A**** (net-crossings, structural rally-state, `[verified]` built, eval-only ? wire to prod = cheapest precision win for G2) and the FusionSegmenter feature path.

59 - ****Pure-precision cleanup**** (****G2****): `rally\_gate` Gate A kills single-feed / held-shuttle ****starts* structurally** (?0?1 net crossings vs a real rally's several) ? the cheapest G2 win and independent of the person stack.

60

61 ****Reuse map (what's new vs there):**** `rally\_gate.py::gate\_windows` (Gate A, built) · `tracknet\_runner` R4 end-trim (extend, don't replace) · `backend/eval/windowing.py` (preset knobs) · person producer in `yolo\_hybrid` (built) · `shuttle\_player\_colocation` (new feature) · `scratch/ingest\_golden.py` + `scratch/boxhill17\_score\_golden.py` (the per-video golden scoring loop, `[verified]`).

62

63 **## 6. Honest limits ? what this does NOT do**

64 - ****n is small.**** One golden clip scored so far (GX010141); the corpus is n=12 with several weak clips. Per-video conclusions firm up only as the owner labels more (GX010137 / GX020140 / a large-count clip next).

65 - ****Nothing here is shipped or even started**** ? implementation is owner-gated. A DONE row in §7 means ****measured on golden*, not *flipped on by default****.

66 - ****Player-state levers need GPU person-detection on the golden clips**** (not yet extracted); until then Lever A/B are design, not data.

```

67     - The boundary tightness is from clips where local *hits* the rally; it says nothing
about the rallies it misses (G1).
68
69     ## 7. Deliverables & progress tracker ? **source of truth**
70     Legend: ? Todo · ? In progress · ? Done · ? Gated. One small PR per row, off
`origin/master`, no stacking.
71
72     | ID | Deliverable | Depends on | Gated? | Status | PR |
73     |----|-----|-----|-----|-----|----|
74     | P0 | Golden scoring loop (`ingest_golden.py` + `boxhill17_score_golden.py`); GX010141
ingested (#12) + scored | ? | No | ? | this work |
75     | P1 | Ingest the rest of the Boxhill golden batch (137/020140 + a large-count clip) ?
per-video baseline table | owner labels | ? owner | ? | ? |
76     | P2 | Wire `rally_gate` **Gate A** (net-crossings) to the served path ? cheapest G2 win
| ? | ? owner | ? | ? |
77     | P3 | Extract person-velocity + `shuttle_player_colocation` on the golden clips
(default-OFF features) | P1 | ? owner | ? | ? |
78     | P4 | **Lever A**: player-state rally-END rule (R-next) ? measure ?F1 / ?merge-rate /
?|?end| vs R4 | P3 | ? owner | ? | ? |
79     | P5 | **Lever B/C**: recall + complementary fusion ? measure recall lift, no precision
regression | P3 | ? owner | ? | ? |
80     | P6 | Launch decision: milestone + best levers default-ON, with a Launch Report (both
rulers + over-seg guard) | P4,P5 | ? owner | ? | ? |
81     | P7 | **Long-rally (>5s) quality lens** (eval `--min-duration`/`--stratify`) + reel
knob (`stitching.min_rally_duration`), both default-OFF. **Measured n=15:** single-court over-
fire is short-FP-dominated (seg_ratio 1.43?1.04) but **multicourt over-fires 1.58× even on long
rallies** (? court-localization, Lever C) and strict tolP doesn't rise (over-merge-
contaminated). See [QUALITY_ITERATIONS.md](QUALITY_ITERATIONS.md) "P7". | ? | No (eval+cfg,
default-OFF) | ? measured | this PR |
82
83     ## 8. Open questions ? owner / external
84     1. **Which lever first** once labels land ? the no-model wins (Gate A wiring + shuttle-
invisibility end-anchor), or go straight to person-velocity? *(decides: owner, after the per-
video baseline)*
85     2. **Per-video target** ? "at-par-with-Gemini-vs-golden on every clip," or an absolute
F1 floor? *(decides: owner)*
86     3. **Recall lever** ? detector-sensitivity tuning vs ensemble fusion as the primary path
for G1. *(decides: data, post-P3)*
87
88     ## 9. Definition of done
89     - [ ] Golden corpus covers the Boxhill set + enough variety that per-video reads are
trustworthy (n meaningfully > 12).
90     - [ ] The **rally-END quick-return** failure mode (G3) is closed: measured ? merge-rate
+ ? |?end| on the clips that exhibit it, no regression elsewhere.
91     - [ ] Local is **at-par-or-better than Gemini vs golden on every corpus video** (no per-
video F1 regression).
92     - [ ] The winning levers are flipped **default-ON** via a Launch Report (over-seg guard
clean on both rulers).
93     - [ ] ? flip `Status ? DONE`, archive to `docs/archives/`.
94
95     ## 10. References
96     **Code anchors `[verified]`**: **`backend/pipeline/detectors/rally_gate.py` (Gate A) ·
`backend/pipeline/detectors/tracknet_runner.py` (R4 end-trim, `_trim_inactive_tail`) ·
`backend/eval/windowing.py` · `backend/eval/rally_seg_eval.py` + `tolerance_metrics.py` (R2) ·
`backend/pipeline/segmenters/yolo_hybrid.py` (person cue) · `scratch/ingest_golden.py`,
`scratch/boxhill17_score_golden.py`.
97     **Internal docs**: **`[RALLY_QUALITY_RESEARCH.md]` (RALLY_QUALITY_RESEARCH.md),
`[MULTI_SIGNAL_FUSION_PLAN.md]` (MULTI_SIGNAL_FUSION_PLAN.md) §4,
`[QUALITY_ITERATIONS.md]` (QUALITY_ITERATIONS.md),
`[EXPERIMENT_HARNESS.md]` (EXPERIMENT_HARNESS.md), `[GOLDEN_VIDEOS.md]` (GOLDEN_VIDEOS.md),
`[HOW_RALLY_DETECTION_WORKS.md]` (HOW_RALLY_DETECTION_WORKS.md).
98
99     ---
100
101     ### Changelog

```

```

102 - 2026-06-18 ? Created. Seeded with the GX010141 golden scoring (first ground-truth
clip), the three named gaps (recall / precision / rally-END), and the player-state end-rule
design from the owner's quick-return insight.
103 - 2026-06-18 ? Added GX010137 (golden #13, normal match play): local F1@0.5 **0.82**
(recall **0.88 > Gemini 0.79**) vs Gemini 0.89. Confirms **opposite failure modes** (local high-
recall/low-precision, Gemini the reverse) ? **precision is local's #1 lever**, fusion the
durable win. Flagged the GX010141 pre-annotator-fix data caveat.
104 - 2026-06-18 ? **P7 long-rally (>5s) quality lens + reel knob (both default-OFF),
measured n=15.** Eval `--min-duration`/`--stratify` (`rally_seg_eval.py`) + product
`stitching.min_rally_duration`. Finding: single-court over-fire is short-FP-dominated (seg_ratio
1.43?1.04) but **multicourt over-fires 1.58x even on long rallies** (? needs court-localization,
Lever C) and strict tolP is gated by over-merge contamination, not just FPs. Sharpened §3 +
added §7 row P7.
105

```

10. assistant (2026-06-19T15:46:47.761Z)

Now let me read the remaining three docs.

11. user (2026-06-19T15:46:49.180Z)

```

1 # A signal-fusion framework for rally detection ? analyzers, scopes & reconcilers
2
3 **Status:** **DESIGN ? owner-gated, NOT started.** A peer to
4 [EXPERIMENT_HARNESS.md](EXPERIMENT_HARNESS.md) and
[MULTI_SIGNAL_FUSION_PLAN.md](MULTI_SIGNAL_FUSION_PLAN.md).
5 This doc specs a small **framework** for evolving rally-detection quality: producers
that emit **signals**
6 at three temporal **scopes** (frame / window / session), a **scorer** that *learns to
weight* them, and an
7 auditable **reconciliation** pass that lints the decision against the evidence. The goal
is to keep piling
8 on clever cues **without the pipeline becoming a pile of special cases.** Docs-only; no
implementation
9 code. Each phase below is a separate, owner-gated PR.
10
11 > **Companion docs:** the detection-layer producers (shuttle / person / audio / shuttle-
state) live in
12 > [MULTI_SIGNAL_FUSION_PLAN.md](MULTI_SIGNAL_FUSION_PLAN.md) (`RallyCue` = swappable
producer; the
13 > `_enrich_candidate_stats` hook; scale/translation-invariant feature discipline, §3.2);
the no-regression
14 > A/B + LOVO measurement these ride is [EXPERIMENT_HARNESS.md](EXPERIMENT_HARNESS.md)
(`Variant`, `compare`,
15 > `compare_unlabeled`); the scorer is the distilled logistic regression (**ADR-004**,
16 > `backend/eval/classifier.py`); the modular rally layer is **ADR-007**
17 > ([archives/decisions/DECISIONS.md](archives/decisions/DECISIONS.md)); the 2-layer
mental model is
18 > [HOW_RALLY_DETECTION_WORKS.md](HOW_RALLY_DETECTION_WORKS.md); the live quality log is
19 > [QUALITY_ITERATIONS.md](QUALITY_ITERATIONS.md).
20
21 > **Framing.** Rally detection is a genuinely hard temporal problem ? weak, multi-modal
cues; interval-level
22 > ground truth that is scarce and expensive; a confound (warm-up / knock-ups / multi-
court / held-shuttle)
23 > that looks identical to real play in any single frame or frequency band. This doc is
written as a
24 > **reusable framework**, not a badminton-specific patch: the contracts are sport-
agnostic and the design
25 > is meant to generalize to other weakly-supervised temporal-segmentation problems
(§10). Where that
26 > ambition touches research claims, §10 says plainly what is and isn't yet
substantiated.
27
28 ---

```

```

29
30     ## 1. The spine ? NO special-casing in the decision path
31
32     This is the one principle everything else serves:
33
34     > **Every producer (analyzer, detector, reconciler) emits a SIGNAL carrying a value + a
confidence/presence.
35     > The SCORING MODEL consumes those signals and *learns to weight them*. We do NOT add
hand-coded `if/else`
36     > branches into the decision.**
37
38     **Producers propose; the scorer arbitrates; the experiment harness + golden set decide
what gets promoted.**
39     That separation is what lets us add a net-hit service fault, a let-cord, a double-
bounce, a shuttle-out
40     cue, a *warm-up span*, a *shuttle-state* ? each as *"register a producer + a feature
column,"* never **add
41     a branch.** A signal that doesn't earn its keep stays at weight ? 0; it costs nothing
because it is an
42     unused column, not a live `if`.
43
44     The pattern already exists in miniature: `experiment.predict()`
(`backend/eval/experiment.py:213`) treats
45     `net_crossings` and `players_in_court` as **feature columns** ? `_gate_mask` is the
trivial scorer
46     (threshold a column), `LogisticRegression` (ADR-004) the richer learned scorer (weight
all columns), a
47     future scorer richer still. The decision path is *signals ? scorer*, with **zero** `if
service_fault:
48     drop`. Analyzers feed it more columns; reconcilers run *outside* it as an audited post-
pass.
49
50     ...
51     DETECTION ? frame-scope producers (expensive, GPU/WSL, run ONCE)
52     WASB shuttle (x,y,v,vis) . person boxes+tracks . audio hits . shuttle-
state(in_air/contact/inactive)*
53                                     ? per-frame-aligned signals (+ net estimate, window
bounds)
54                                     ?
55     shuttle-seeded candidate windows (FusionSegmenter, P2)
56                                     ?
57     ANALYZERS ? decision-layer producers (pure, offline, re-scorable) [forward
seam]
58     ?? window-scope (WindowAnalyzer): one candidate ? features + events + boundary
hints e.g. service-fault
59     ?? session-scope (SessionAnalyzer): whole timeline ? labeled spans + per-window
membership e.g. warm-up
60                                     ? every signal ? candidate_segments.stats as feature
columns
61                                     ?
62     SCORING / DECISION ? signals ? learned scorer (weights, NO branches)
63     _gate_mask (trivial) + LogisticRegression (ADR-004) + harness-pinned model
64                                     ? predicted rally segments
65                                     ?
66     RECONCILERS ? backward seam, "linter for rally decisions" (report-first, idempotent)
67     ConsistencyRule registry: decision-vs-evidence conflicts ? Issue + corrected set
68                                     ?
69     stats . events . spans . corrections record . highlights EXPERIMENT HARNESS
gates EVERY promotion
70     (* shuttle-state = a richer frame detector; its PRODUCER lives in
MULTI_SIGNAL_FUSION_PLAN, consumed here ? §3.5)
71     ...
72
73     ---
74

```

```

75     ## 2. The framework ? signals, scopes, and who arbitrates
76
77     The architecture has exactly three kinds of moving part. Capability is added by
78     registering a producer;
79     the decision logic never grows a branch.
80
81     **Producers emit signals at three temporal SCOPES.** A signal at a coarser scope is
82     computed by aggregating
83     finer-scope signals; everything ultimately lands as a per-candidate feature column (or
84     an event/span the
85     scorer or a human consumes).
86
87     | Scope | Producer | Sees | Emits | Cost | Examples |
88     |---|---|---|---|---|---|
89     | **frame** | detector (`RallyCue`) | raw frames / audio | per-frame signal |
90     **expensive** (GPU/WSL) | WASB shuttle, person boxes, audio hits, **shuttle-state** (§3.5) |
91     | **window** | `WindowAnalyzer` (§3.1) | one candidate window's aligned frame-signals |
92     features (+conf), events, boundary hints | cheap, pure | service-fault, double-bounce, let-cord,
93     shuttle-out |
94     | **session** | `SessionAnalyzer` (§3.4) | the whole timeline of windows | labeled spans
95     + per-window membership features | cheap, pure | **warm-up / practice**, game/break phases |
96
97     **The scorer arbitrates.** It is the only place a decision is made, and it makes it by
98     *weighting columns*,
99     never branching. Today: `_gate_mask` (threshold) + `LogisticRegression` (learned
100     weights, ADR-004). The
101     confidence column on every signal lets the scorer discount an uncertain producer instead
102     of trusting a
103     binary verdict ? the small-N-safe way to fold in a deterministic-ish cue.
104
105     **Reconcilers audit.** After the scorer decides, the backward seam (§6) lints the
106     decision against the
107     persisted evidence and proposes corrections ? report-first, never silent.
108
109     **Four invariants make this safe to grow** (the load-bearing design claims):
110     1. **Signal, not verdict** ? every producer's output is a weighted feature; the decision
111     path has no
112     producer-specific branch. ? unbounded cues without combinatorial special-casing.
113     2. **Expensive detection ? cheap decision** ? frame-scope detection runs once and is
114     persisted; window/
115     session signals and the scorer are pure re-scoring (EXPERIMENT_HARNESS §2). ?
116     experiments are offline,
117     deterministic, zero production risk.
118     3. **Report-first reconciliation** ? corrections are recorded and reviewable, never
119     applied silently. ?
120     human-in-the-loop, auditable.
121     4. **Promote only on measured held-out lift** ? LOVO on the golden set + the
122     `run_regression.py` gate;
123     default OFF; rollback = config flip. ? no-regression evolution.
124
125     ---
126
127     ## 3. The forward seam ? injectable analyzers
128
129     Pluggable producers ? registered like `segmenter_registry`
130     (`backend/pipeline/segmenters/base.py:35`) ?
131     that run over the **per-frame-aligned signals already produced**: the WASB shuttle
132     trajectory
133     (`.frame/.visible/.x/.y` + velocity), the person boxes per frame
134     (`PersonDetector.detections_per_frame`,
135     `person_detector.py:256`), the net line (`rally_gate.estimate_play_axis`, camera-
136     agnostic), and the window
137     bounds. Analyzers are **pure** ? no I/O, no torch/cv2 ? mirroring `fusion_features.py`'s
138     discipline, so
139     their math is unit-testable in the GPU-free dev container.

```

```

119
120     ### 3.1 Contracts (JSON-serializable; proposed `backend/pipeline/analyzers/`)
121
122     ```python
123     @dataclass(frozen=True)
124     class Signal:
125         value: float; confidence: float = 1.0    # persisted as `<producer>__<key>` (+
never a verdict    `_confidence`), [0,1]
126
127     @dataclass(frozen=True)
128     class AnalyzerEvent:
129         type: str; timestamp: float; confidence: float = 1.0    # "net_hit"/"double_bounce"/?
; src-absolute s
130         player: Optional[str] = None; details: Dict[str, Any] = field(default_factory=dict)
131
132     @dataclass(frozen=True)
133     class BoundaryHint:
134         kind: Literal["start", "end"]; timestamp: float; confidence: float; reason: str
135
136     @dataclass(frozen=True)
137     class AnalyzerResult:
138         signals: Dict[str, Signal] = field(default_factory=dict)
139         events: List[AnalyzerEvent] = field(default_factory=list)
140         boundary_hints: List[BoundaryHint] = field(default_factory=list)
141
142     @dataclass
143     class WindowSignals:
144         start: float; end: float; fps: float; frame_width: float; frame_height: float
145         shuttle: List["TrajectoryPoint"]    # in-window points (.frame/.visible/.x/.y)
146         persons: Dict[int, List[Tuple[int, BBox]]    # frame_idx ? [(track_id, box)]; {} when
person cue OFF
147         net: Tuple[str, float]
148         frame_states: Dict[int, "ShuttleState"]    # frame_idx ? state; {} until the
shuttle-state detector (§3.5)
149         base_stats: Dict[str, float]    # motion stats already computed for the window
150
151     class WindowAnalyzer(ABC):
152         name: str; version: str
153         feature_keys: List[str]; event_types: List[str]    # declared columns/events (for
the harness)
154         @abstractmethod
155         def analyze(self, w: WindowSignals) -> AnalyzerResult: ...
156
157     The three roles an `AnalyzerResult` can fill:
158
159     | Role | Sink | Consumed by |
160     |---|---|---|
161     | *(a) features* (with confidence) | `candidate_segments.stats` as `<analyzer>__<key>`
(+ `_confidence`) | the scoring model **and** the harness, as feature columns (§4) |
162     | *(b) events* | the `events` table (buffered on the candidate, flushed on
confirmation, §5) | history / highlights / audit |
163     | *(c) boundary hints* (optional) | stashed in `stats`; never auto-applied | the
reconciler (§6) / a future boundary-aware scorer |
164
165     ### 3.2 Worked example ? the SERVICE-FAULT analyzer (window-scope)
166
167     The owner's held-shuttle bug has a sibling: a **service fault** where the serve hits the
net and the
168     shuttle comes to rest in the net region. Pure trajectory, GPU-free, no person cue

```

```

needed:
170
171     - **Input:** the window's shuttle trajectory + the `rally_gate` net estimate.
172     - **Logic:** the shuttle **enters the net region** (|coord ? net| < deadband) **and**
its velocity
173     **decays to ? 0** there (comes to rest) ? high-probability net-hit service fault.
Confidence scales with
174     how cleanly the halt sits inside the net band.
175     - **Emits:** a `service_fault` **signal** (value = presence, confidence = halt
cleanliness) + a `net_hit`
176     **event** at the halt timestamp + a rally-**END** `BoundaryHint` at that timestamp.
177
178     Crucially: the analyzer does **not** drop the window. It proposes `service_fault`; the
scorer sees
179     `service_fault` + `service_fault_confidence` as two more columns and **learns**
whether/how much to
180     down-weight. The boundary hint is an audited proposal the reconciler may act on ? never
a branch.
181
182     ### 3.3 Injection point ? generalize `_enrich_candidate_stats`
183
184     The FusionSegmenter hook (`fusion_hybrid.py:70`) already turns "compute fixed features"
into per-window
185     enrichment. We generalize it to ***run each enabled analyzer*** ? additively, so the
existing
186     `net_crossings`/person paths are unchanged:
187
188     ```python
189     def _enrich_candidate_stats(self, cw, points, fps, frame_width, video_path, idx_cfg):
190         stats = super()._enrich_candidate_stats(...)          # base motion keys
191         stats["net_crossings"] = ...                          # Gate-A signal (kept)
192         # ? person features when the person cue is on (kept) ?
193         w = self._window_signals(cw, points, fps, frame_width, video_path, idx_cfg)
194         for analyzer in analyzer_registry.enabled(idx_cfg):   # NONE enabled ? byte-identical
to control
195             res = analyzer.analyze(w)
196             for key, sig in res.signals.items():
197                 stats[f"{analyzer.name}_{key}"] = sig.value
198                 stats[f"{analyzer.name}_{key}_confidence"] = sig.confidence
199                 stats.setdefault("_events", []).extend(e.__dict__ for e in res.events)      #
flushed in Phase 4
200             stats.setdefault("_boundary_hints", []).extend(h.__dict__ for h in
res.boundary_hints)
201         return stats
202     ```
203
204     Flag-gated, **default OFF** via the existing `indexing.fusion.cue_flags` (a sibling
`analyzer_flags`
205     map keyed by `analyzer.name`). With every analyzer OFF, the loop is empty ? candidate
rows are
206     byte-identical to the substrate ? the no-regression invariant holds (§7).
`net_crossings` and the person
207     features are the **prototype analyzers** the seam retrofits first (proving it with zero
behavior change);
208     every new cue after that is a pure add.
209
210     ### 3.4 Session-scope analyzers ? the WARM-UP / PRACTICE example
211
212     Some structure is invisible at the window scope. **warm-up** looks exactly like a rally
in any single
213     window ? shuttle in flight, two players, net crossings, sustained motion ? which is
*why* it is a notorious
214     false-positive source. It is only distinguishable by **timeline structure**: a long
contiguous active
215     stretch *without* the rally?dead-time?rally cadence of scored play, usually (not always)

```

```

at the start.
216
217     So warm-up needs a producer that sees the whole sequence of windows, not one window.
That is a sibling
218     tier ? a `SessionAnalyzer` that runs once per video (a second pass, after per-window
enrichment) and emits
219     a labeled span plus a per-window membership feature:
220
221     ```python
222     @dataclass(frozen=True)
223     class Span:
224         kind: str                # "warmup" | "practice" | "break" | "game" | ?
225         start: float; end: float; confidence: float; reason: str
226
227     @dataclass(frozen=True)
228     class SessionResult:
229         spans: List[Span] = field(default_factory=list)
230         # window_index ? signals merged back into THAT window's stats (e.g. {"in_warmup":
Signal(0.8, 0.6)})
231         window_features: Dict[int, Dict[str, Signal]] = field(default_factory=dict)
232
233     class SessionAnalyzer(ABC):
234         name: str; version: str
235         span_kinds: List[str]; feature_keys: List[str]
236         @abstractmethod
237         def analyze(self, windows: List[WindowSignals], session: "SessionContext") ->
SessionResult: ...
238     ```
239
240     WarmupAnalyzer (worked):
241     - v0 (free, no new model): find the leading contiguous high-activity span with
low inter-window
242     dead-time and no serve-reset cadence, over the persisted window timings we already
have. Emit
243     `Span(kind="warmup", ?)` + per-window `in_warmup` = overlap fraction of the window
with the span.
244     - v1 (learned): fed by the shuttle-state cadence (§3.5) / the audio thwack
cadence ? warm-up =
245     continuous hits without the point-ending pause rhythm; generalizes to mid-session
practice / between-
246     game knock-ups (drop the position prior).
247
248     You framed the consumption exactly right: "so the scorer may consume it."
in_warmup is a feature
249     column; the scorer learns to down-weight windows inside the span. Warm-up is the
poster child for
250     no-special-casing: the naive version is `if t < warmup_end: drop` ? a hand-coded
branch hanging off a
251     fuzzy boundary; a 20-second error deletes a real opening rally. As a weighted
feature, a wrong boundary
252     just slightly mis-weights a couple of windows the other cues rescue. The span is also
persisted as
253     evidence a reconciler (§6) may use to down-weight a whole cluster, but the primary path
is feature ? scorer.
254
255     It is measurable today with no new labels: the golden labels already treat warm-up
knock-ups / pre-serve
256     as dead time = negatives (Roora starts a rally at serve-contact; fusion §1 names
"warm-up knock-ups" as
257     a precision target). So an `in_warmup` feature is directly A/B-able on the 6 golden
videos for precision
258     lift ? that is the promotion gate.
259
260     3.5 Consuming a richer detector ? SHUTTLE-STATE (frame-scope producer)
261

```

```

262     A natural new **detector**: a per-frame shuttle-state classifier ? `in_air` /
`racket_contact` /
263     `inactive`. Note the scope: it **processes frames ? a new per-frame signal**, like WASB,
so it is a
264     **frame-scope producer in the detection layer** ? its spec belongs in
265     [MULTI_SIGNAL_FUSION_PLAN.md](MULTI_SIGNAL_FUSION_PLAN.md) (it is a new `RallyCue`),
**not** a `WindowAnalyzer`.
266     This doc records **how the framework consumes it** once it exists:
267
268     - a **`WindowAnalyzer`** turns the persisted per-frame state into window features ?
`racket_contact_count`
269     (? shot cadence), `inactive_ratio` (? dead-time) ? that flow into the scorer as
columns;
270     - a **`SessionAnalyzer`** (warm-up, §3.4) reads its cadence;
271     - the **reconcilers** (§6) get *stronger* evidence: `trailing_dead_time` / `over_merged`
currently infer
272     inactivity from a velocity threshold; a learned `inactive` state is a more direct,
occlusion-robust
273     evidence source for "truncate / split here."
274
275     **Stage it (don't build the model first).** Most of the state is derivable for free from
signals we already
276     have ? `in_air` vs `inactive` ? WASB velocity + visibility; `racket_contact` ?
trajectory **direction
277     reversals** (fusion §4: "each reversal ? a shot") **and** the scaffolded **audio
thwack**
278     (`backend/pipeline/audio/hit_detector.py`, ADR-007's #1 cue). So **v0 = a derived
`shuttle_state` analyzer**
279     over existing signals (no new model); measure it. **The dedicated per-frame vision model
earns its keep
280     only where v0 hits a ceiling: WASB trajectory gaps** ? occlusion, motion blur, fast
smashes ? where there
281     is no `(x,y)` to threshold but an appearance model could still call `racket_contact`.
**Label blocker:** we
282     have **zero** per-frame state labels (golden labels are rally intervals); a dedicated
model needs weak/
283     distilled labels or a new (minors-gated) annotation task ? the same gate as feature-
level fusion. If you can
284     weak-label state from existing signals, prefer the derived analyzer until the gap-
filling case is proven.
285
286     ---
287
288     ## 4. How signals reach the scorer ? columns, not branches
289
290     The persisted `stats` columns are exactly what the harness already lifts.
`load_video_candidates(...,
291     feature_names=[...])` (`experiment.py:543`) reads each name out of the row's `stats`
JSON into a feature
292     column; `predict()` (`experiment.py:213`) gates/scores on those columns and merges. So a
new producer's
293     signal becomes a scorer input by **adding its column name to a `Variant`'s
`feature_names`** ? no new
294     scoring code, no branch:
295
296     ```python
297     # A learned Variant that lets the scorer weight the new signals (service-fault + warm-up
membership):
298     fusion_plus = Variant(name="fusion+analyzers",
299                           feature_names=[*SHUTTLE_FEATURES, "person__players_in_court",
300                                           "service_fault__service_fault",
"service_fault__service_fault_confidence",
301                                           "warmup__in_warmup",
"warmup__in_warmup_confidence"])
302     compare(golden_videos, CONTROL, fusion_plus)          # LOVO-honest B ? A on the 6 golden

```

```

videos
303     ``
304
305     The model **learns the weights** ? down-weight a window with a high-confidence
`service_fault` or high
306     `in_warmup`; up-weight ?2 in-court players + multiple net-crossings. **What the scorer
cannot see, it
307     cannot special-case; what it can see, it weights from data.** Same logistic regression
as ADR-004; we add
308     columns, never a net.
309
310     ---
311
312     ## 5. Persistence
313
314     - **Features ? `candidate_segments.stats`** (`database.py:240`). Namespaced
`<producer>__<key>`
315     (+ `_confidence`). Persisted **as a shadow** even when the served decision ignores
them, so any future
316     `Variant` re-scores offline (EXPERIMENT_HARNESS §5.2). `version`/`name` recorded for
provenance.
317     - **Spans ? window membership + a session record.** A `SessionAnalyzer`'s per-window
`in_<span>` features
318     ride in each window's `stats` (so they re-score like any feature); the span list
itself (start/end/kind/
319     confidence) is a small per-video record (a JSON sidecar first; a table only if it
grows).
320     - **Events ? the `events` table** (`database.py:227`, `add_event(segment_id, ts, type,
player, details)`).
321     Its FK is to `play_segments`, which exist only **after** the AI handoff confirms a
candidate. So
322     analyzer events are **buffered on the candidate** (`stats["_events"]`) and **flushed**
to `events` in
323     Phase 4, when `_candidate_id` ? `segment_id` is linked (`trajectory_hybrid.py:283`).
Candidate-level
324     discoveries survive for offline re-scoring; the `events` table stays "events on
confirmed segments."
325     - **Corrections record** (§6) ? report-first means corrections are **recorded, never
silent.** Phase one:
326     a JSON sidecar shaped like `experiment_leaderboard.json` (one row per issue: segment,
kind, evidence,
327     before?after, reason, timestamp). When auto-apply is promoted, add a `corrections`
table via the
328     `PRAGMA user_version` migration ladder (`database.py:88`, `if version < 3:`) so
applied corrections are
329     queryable and reversible.
330
331     ---
332
333     ## 6. The backward seam ? the reconciliation pass (a linter for rally decisions)
334
335     Given **predicted segments + the persisted per-frame/per-window/per-session signals**,
detect where the
336     **DECISION contradicts the EVIDENCE** and propose corrections. This is the backward dual
of the analyzers:
337     analyzers add evidence *before* the decision; reconcilers audit the decision *against*
evidence *after* it.
338
339     ### 6.1 Contracts (proposed `backend/pipeline/reconcilers/`)
340
341     ```python
342     @dataclass(frozen=True)
343     class Issue:
344         segment_index: int; kind: str          # kind = rule id, e.g. "trailing_dead_time"
345         evidence: Dict[str, Any]              # the signal values that triggered it

```

```

(auditable)
346     before: Interval; after: Optional[List[Interval]]    # None=drop · [iv]=corrected ·
[iv1,iv2]=split
347     reason: str; confidence: float
348
349     @dataclass(frozen=True)
350     class Correction:
351         issues: List[Issue]; corrected: List[Interval]      # the FULL corrected set
(idempotent)
352
353     class ConsistencyRule(ABC):
354         name: str; precedence: int          # FIXED ordering; lower runs first
355         @abstractmethod
356         def check(self, seg: Interval, signals: "SegmentSignals") -> Optional[Issue]: ...
357
358     def reconcile(segments, signals, rules) -> Correction: ...    # fixed precedence;
idempotent
359     ```
360
361     ### 6.2 Worked rules (fixed precedence)
362
363     | # | Rule | Evidence contradiction | Correction |
364     |---|---|---|---|
365     | 1 | `trailing_dead_time` | segment still "in rally" at its end, but the shuttle went
inactive earlier | truncate to the last-active frame *(the owner's misfire fix)* |
366     | 2 | `no_net_crossing` | `net_crossings < 1` ? nothing ever crossed the net | drop
(or flag) |
367     | 3 | `over_merged` | a dead-time gap > `merge_gap` of shuttle-inactivity sits
inside one segment | split at the gap |
368     | 4 | `boundary_slack` | first sustained shuttle activity / first net-crossing is well
inside the start | tighten start to serve-contact |
369     | 5 | `inside_warmup` | the segment lies inside a high-confidence `warmup`/`practice`
span (§3.4) | flag (drop only under apply-flag) |
370
371     Each rule reads only persisted evidence (trajectory activity per frame,
`net_crossings`, person
372     occupancy, analyzer signals/events, session spans, shuttle-state) ? it is a
deterministic linter, not a
373     re-detector. As richer producers land (shuttle-state, warm-up), rules get stronger
evidence, not new
374     branches in the decision path.
375
376     ### 6.3 Report-first, apply-optional (non-negotiable)
377
378     `reconcile()` returns both an issues report and a corrected set. It NEVER
silently rewrites
379     (auditability + the repo's honesty/attribution working agreement). The pipeline either:
380
381     - (default) persists the report and surfaces it for human review ? production
output unchanged; or
382     - (flagged, `reconcile.apply`) applies the corrected set, recording every change
in the
383     corrections record (before?after, rule, evidence). Rollback = flip the flag.
384
385     It is a `Variant` transformation, so its lift is A/B-measurable BEFORE auto-apply. A
reconciler is a
386     pure post-`predict` stage: `reconcile(predict(...), signals, rules).corrected`. Wrapping
that as a
387     `RECONCILED` variant lets `compare(CONTROL, RECONCILED)` report the LOVO lift on the 6
golden videos
388     ([QUALITY_ITERATIONS.md](QUALITY_ITERATIONS.md) corpus) ? promote to auto-apply only
on measured lift.
389
390     Idempotent; fixed rule precedence. `reconcile(reconcile(x)) == reconcile(x)`
(truncating an

```

```

391     already-truncated segment is a no-op)); precedence is a fixed table so the result is
deterministic and
392     order-independent of input segment order.
393
394     ---
395
396     ## 7. No-regression discipline (identical to the harness contract)
397
398     The same four guarantees EXPERIMENT_HARNESS.md §6 makes ? applied to every producer and
the reconciler:
399
400     - **Default OFF.** Analyzers gate on `analyzer_flags` (default empty); the reconciler is
report-first
401     (apply gate default off). Merged code **cannot** change served behavior until
explicitly enabled.
402     - **Shadow-persist, then promote.** Signals/spans/issues are persisted even while
ignored, so a `Variant`
403     re-scores them offline; a signal becomes a *decision input* only when a Variant that
references it wins
404     LOVO. (EXPERIMENT_HARNESS §5.2's shadow mode.)
405     - **Promotion only via the eval gate.** `run_regression.py` against the golden
`baseline.json` (exit 1
406     blocks); the experiment leaderboard records the spec hash + per-video + LOVO deltas +
label-set hash.
407     - **Rollback = flip a config flag, never `git revert`.** The pinned `CONTROL` variant is
always present.
408
409     The keystone test (extends the harness's existing invariant): **all analyzer flags OFF +
reconciler
410     report-only ? candidate rows AND predictions byte-identical to control.**
411
412     ---
413
414     ## 8. How it composes with fusion P2
415
416     - **Window-analyzers plug into fusion P2's enrichment hook.**
`FusionSegmenter._enrich_candidate_stats`
417     (`fusion_hybrid.py:70`) is the exact injection point §3.3 generalizes; `net_crossings`
and the person
418     features become the first registered analyzers (no behavior change). New cues are add-
a-producer, per
419     ADR-007 / MULTI_SIGNAL_FUSION_PLAN §2.
420     - **Session-analyzers run as a second pass** over the per-window stats the same hook
persists.
421     - **The shuttle-state detector is a new fusion `RallyCue`** (frame-scope producer) ? it
belongs in the
422     fusion plan; this framework only *consumes* it (§3.5).
423     - **Reconcilers consume fusion P2's persisted signals** ? the same
`candidate_segments.stats` plus the
424     persisted trajectory; no new detection.
425     - Everything inherits fusion P2's control-reproducing defaults, so the fusion segmenter
with all flags off
426     still seeds and hands off identically to `wasb_hybrid`.
427
428     ---
429
430     ## 9. Trade-offs & risks
431
432     - **Deterministic vs learned signals.** A producer may emit a verdict-shaped signal
(`service_fault=1.0`),
433     but we persist it as a **feature**, so the scorer still arbitrates. Risk: a
confidently-wrong cue.
434     Mitigation: the **confidence column** + the **LOVO gate** ? a bad signal earns ? 0
weight, never promotes.
435     - **The temporal confound is the hard core.** Warm-up-vs-play, multi-court, and held-

```

```

shuttle all look
436     identical in any single frame / frequency band ? QUALITY_ITERATIONS' standing lesson:
"the dead-time
437     confusion is rally-vs-non-rally activity in the same place and frequency band... only
the learned model's
438     temporal features crack it." This is *why* warm-up is a **feature the model weights**,
not a heuristic
439     span-cutter, and why session-scope structure earns its own tier.
440     - **Scope discipline.** Put a producer at its *natural* scope: shuttle-state is frame-
scope (a detector),
441     not a window analyzer; warm-up is session-scope, not per-window. Mis-scoping forces a
producer to either
442     recompute detection or miss the structure it needs.
443     - **Ordering / idempotence (reconcilers).** Fixed precedence table + an idempotence test
444     (`reconcile?reconcile == reconcile`).
445     - **Signal, not verdict.** A producer that *decides* (returns keep/drop) re-introduces
special-casing;
446     reviews must reject "analyzer returns a boolean keep/drop."
447     - **Report-first auditability.** Reconcilers never silently mutate output; every applied
change is recorded.
448     - **Small-N feature discipline.** Few, scale/translation-invariant columns
(counts/ratios/presence, never
449     raw coords ? MULTI_SIGNAL_FUSION_PLAN §3.2); the model stays a logistic regression,
not a net.
450     - **Labels gate the learned producers.** Shuttle-state and feature-level fusion need
labels we don't yet
451     have; warm-up is measurable *now* because golden treats it as negatives. Know which
producers are gated.
452     - **Cost & licensing.** Pure analyzers are GPU-free; person- and shuttle-state-dependent
producers pull
453     torch / a second WSL pass ? gate behind their cue flags; all deps stay MIT/Apache/BSD
(ADR-002/005).
454     - **Event-table timing.** The buffer-then-flush wiring (§5) is the one non-obvious
integration detail.
455
456     ---
457
458     ## 10. Generality & research positioning
459
460     This is written as a **framework**, not a badminton patch. Read sport-agnostically it
is: *a registry-based,
461     multi-scope **signal-fusion** architecture for **weakly-supervised temporal
event/segment detection**, with
462     a learned arbiter and an **auditable post-hoc correction** layer.* The pieces that
generalize:
463
464     - **Multi-scope producers ? one feature table ? a learned arbiter.** The
frame/window/session taxonomy and
465     the "signal-not-verdict" rule are domain-independent. Any temporal problem with
**weak, multi-modal cues
466     at different time scales** (broadcast sports event spotting, surgical-phase
recognition, call-center /
467     meeting segmentation, activity logs) fits the same shape: add a producer, add a
column, let the arbiter
468     weight it, gate promotion on held-out lift.
469     - **Detection ? decision (persist once, re-score forever).** Decouples the expensive,
hard-to-reproduce
470     perception from the cheap, deterministic decision ? the property that makes the whole
thing
471     experimentally honest and cheap to iterate.
472     - **Auditable reconciliation as a first-class layer.** Most pipelines bury post-hoc
fixes as silent
473     if-branches; treating consistency-vs-evidence as a *report-first, A/B-measured* pass
is the transferable
474     methodological idea (and the human-in-the-loop / safety story).

```

475 - ****The honest measuring stick.**** Interval-level, scarce ground truth + ****leave-one-VIDEO-out**** (group by
 476 match, never a held-out window from the same video) is the discipline any small-corpus
 temporal study
 477 needs to avoid flattering itself.
 478
 479 ****What is defensibly contributed vs. what is standard.**** Multi-modal ****late fusion****,
 weak supervision, and
 480 learning a linear/logistic arbiter over hand-designed features are all standard. The
combination offered
 481 here ? (a) a uniform producer-registry across ****three temporal scopes****, (b) a hard
****no-branch**** decision
 482 invariant that makes extension monotone, (c) a ****report-first, harness-measured**
 reconciliation****** layer, and
 483 (d) the ****detection?decision persistence**** substrate that makes every experiment a zero-
 risk offline
 484 re-score ? is what would be written up, with rally detection as the motivating hard
 problem.
 485
 486 ****Honesty caveat (attribution working agreement).**** No external results or citations are
 asserted here. A
 487 real write-up still owes: a proper ****related-work survey**** (temporal action
 segmentation/detection, sports
 488 event spotting, multimodal fusion, weak supervision ? to confirm which claims are
 actually novel), ****strong**
 489 **baselines****, ****per-producer ablations****, and reporting on ****>1 sport / camera**** to
 substantiate the
 490 generality claim. Today the corpus is ****6 matches**** (QUALITY_ITERATIONS) ? enough for
 direction-checks and
 491 LOVO sign-tests, ****not**** enough to claim a general model; corpus growth is the highest-
 ROI unblock. Treat
 492 §10 as a ***positioning sketch***, not a results section.
 493
 494 > ****Update (2026-06-14): that related-work survey is now done ? see [§13](#13-prior-art-terminology--course-corrections-literature-review-2026-06-14).****
 495 > The honest finding: the ***skeleton*** (TAL/TAS + late fusion) is standard and must be
****cited, not claimed****;
 496 > the only **`novel-combination`** is the report-first reconciler-as-linter bundle; warm-up-
 as-a-soft-feature is
 497 > the thinnest genuine gap. §13 also lists the verified citations and the prioritized
 course corrections.
 498
 499 ---
 500
 501 **## 11. Phased execution (owner-gated; ONE PR per slice, off `master`, default OFF)**
 502
 503 - ****A0 ? this doc****, indexed in `docs/README.md`, cross-linked from the peers + ADR-007.
***(this**
 504 deliverable; docs-only.)*
 505 - ****A1 ? analyzer contracts + registry****
 (`WindowAnalyzer`/`AnalyzerResult`/`Signal`/`AnalyzerEvent`/
 506 `BoundaryHint` + `analyzer\_registry`); generalize `\_enrich\_candidate\_stats` to iterate
 enabled analyzers
 507 (none enabled ? byte-identical). Retrofit `net\_crossings` + person features as the
 first two analyzers.
 508 - ****A2 ? service-fault analyzer**** behind `analyzer\_flags.service\_fault` (default OFF);
 signal + `net\_hit`
 509 event + end boundary hint; shadow-persisted; offline `compare(CONTROL,
 +service_fault)` LOVO.
 510 - ****A3 ? event flush wiring**** (buffer analyzer events on candidates ? `add\_event` on
 confirmation).
 511 - ****A4 ? `SessionAnalyzer` tier + `WarmupAnalyzer` v0**** (leading-span heuristic over
 window timings),
 512 default OFF; persists `warmup\_\_in\_warmup` + the span record; measured for
****precision**** lift on golden.

```

513     - **A5 ? derived `shuttle_state` analyzer v0** (velocity + visibility + direction-
reversals (+ audio);
514     **no new model**); emits `racket_contact_count` / `inactive_ratio`; measured LOVO.
*(The dedicated
515     per-frame shuttle-state DETECTOR is a separate MULTI_SIGNAL_FUSION_PLAN `RallyCue`
item ? gated on
516     per-frame labels; build only if v0 is blocked by trajectory gaps.)*
517     - **R1 ? reconciler contracts + registry + `reconcile()`**
(`ConsistencyRule`/`Issue`/`Correction`) with
518     the worked rules; pure, idempotent, unit-tested; **not** wired to apply (report-only).
519     - **R2 ? reconciler as a `Variant` transform**: optional post-`predict` stage;
`compare(CONTROL,
520     RECONCILED)` LOVO; report-only in production.
521     - **R3 ? corrections persistence + opt-in apply**: JSON sidecar first, then the
`corrections` table
522     (`user_version < 3`); apply behind `reconcile.apply`; **rollback = flip the flag.**
523
524     Each PR is gated by the no-regression invariant (§7) + `run_regression.py`. Order is a
suggestion; A4/A5/R1
525     are independent and can be reprioritized on owner steer.
526
527     ---
528
529     ## 12. Verification
530
531     - **Offline, runnable in this dev container (no GPU/torch/cv2):** pure analyzer math on
synthetic
532     trajectories + person boxes (service-fault fires on a synthetic net-halt, silent on a
clean rally;
533     `WarmupAnalyzer` v0 finds a synthetic leading knock-up span; derived `shuttle_state`
matches a scripted
534     velocity/visibility sequence); the **no-regression invariant** (all flags OFF +
reconciler report-only ?
535     candidate rows + predictions byte-identical to control); the **reconciler
idempotence** test
536     (`reconcile?reconcile == reconcile`) and fixed-precedence determinism;
`compare(CONTROL, variant)`
537     deterministic on synthetic candidate rows.
538     - **Metrics ? adopt the field's (see [§13](#13-prior-art-terminology--course-
corrections-literature-review-2026-06-14)/C4):**
539     report **segmental F1@{10,25,50} + edit score** (TAS) and **tight/loose mAP@?**
(action spotting) alongside
540     temporal-IoU. tIoU alone is nearly blind to the over-segmentation the reconciler
exists to fix, so a
541     reconciler win ? or an IoU-chasing rule that shreds rally structure ? is invisible
under tIoU.
542     - **Machine-side (owner, GPU/WSL ? full pipeline can't run here):** persist analyzer
signals on a real
543     clip; `compare(CONTROL, +service_fault)`, `compare(CONTROL, +warmup)`,
`compare(CONTROL, RECONCILED)`
544     LOVO on the 6 golden videos; confirm the owner's **trailing-dead-time misfire**
truncates, the
545     **held-shuttle / net-hit service feed** is down-weighted, and **warm-up windows** lose
precision-mass;
546     spot-check `net_hit` events on confirmed segments. **Grade with §13/C4 metrics + per-
fold CIs (C1).**
547     - **Docs (A0):** links resolve; indexed in `docs/README.md`; cross-linked from
548     [MULTI_SIGNAL_FUSION_PLAN.md](MULTI_SIGNAL_FUSION_PLAN.md),
[EXPERIMENT_HARNESS.md](EXPERIMENT_HARNESS.md),
549     [HOW_RALLY_DETECTION_WORKS.md](HOW_RALLY_DETECTION_WORKS.md), and ADR-007.
550
551     ---
552
553     ## 13. Prior art, terminology & course corrections (literature review, 2026-06-14)
554

```

555 A multi-agent web-research pass surveyed the eight design patterns of this framework
against the
556 literature and ****adversarially verified every citation**** (real-paper check + closer-
prior-art hunt).
557 ****Result: zero fabricated papers****; a few wrong author-lines / metadata slips were
caught and are
558 corrected below. This section is the honest "are we reinventing the wheel?" answer and
the actionable
559 course corrections ? all ****owner-gated, NOT yet implemented**** (they fold into the A*/R*
phases of §11).
560
561 **### 13.1 Verdict ? what is standard vs. what is ours**
562
563 > ****TL;DR.**** The architecture's **skeleton** is standard and must be ****cited, not**
claimed**: candidate-windows
564 > + learned scorer = ****Temporal Action Localization**** (proposal-then-score); the
reconciler's
565 > over-segmentation fixes = ****Temporal Action Segmentation**** refinement/post-processing;
"signal not verdict /
566 > learned arbiter over cue columns" = ****late / score-level fusion**** realized as
****stacked generalization**** ?
567 > ~20 years old and **already present in racket sports**. The ****defensible delta** is the
engineering/governance
568 > envelope**: detection?decision persisted as a re-score substrate + a ****report-first,**
A/B-lift-gated,
569 > idempotent, human-in-the-loop reconciler-as-linter** keyed on decision-vs-evidence
contradiction + the
570 > flag-gated no-regression promotion harness, on a deliberately tiny LOVO-by-match
golden set with an
571 > interpretable scorer. Of the patterns, only the ****reconciler-as-linter**** rated `novel-
combination`; the
572 > rest are `partial-overlap`. Park any novelty claim narrowly on (a) that reconciler
bundle and (b)
573 > ****warm-up-as-a-soft-session-feature**** (the one slice with genuinely thin prior art).
574
575 | Pattern (our framing) | Field's name | Closest verified prior art | Verdict |
576 |---|---|---|---|
577 | Candidate-windows + scorer + reconciler-as-refinement | Temporal Action Localization /
Segmentation + boundary refinement | ASRF (arXiv:2007.06866); Activity Grammars
(arXiv:2312.04266); BMN (arXiv:1907.09702) | partial-overlap |
578 | "Signal not verdict" ? learned arbiter over cue columns | Late / score-level fusion +
stacked generalization | Snoek 2005 (10.1145/1101149.1101236); Wolpert 1992; racket-sports A/V
rally-break fusion (CVIU 2009) | partial-overlap |
579 | Detection?decision, "persist once, re-score forever" | Frozen-backbone cached features
+ decoupled proposal/classification | S-CNN (arXiv:1601.02129); CBR (arXiv:1705.01180);
Decouple-SSAD (arXiv:1904.07442) | partial-overlap (? field default) |
580 | Report-first reconciler (contradiction ? audited correction, measured before apply) |
Model assertions + constraint-based repair ("ML linter") | Model Assertions (arXiv:2003.01668);
HoloClean (arXiv:1702.00820); TFDV (MLSys 2019); EventAnchor (arXiv:2101.04954) | ****novel-
combination**** |
581 | Scarce interval labels + tiny LR + LOVO-by-match | Point/timestamp-supervised TAL +
grouped (LOGO/LOSO) CV | SF-Net (arXiv:2003.06845); LOOCV-bias (arXiv:2406.01652); Snorkel
(arXiv:1711.10160) | partial-overlap |
582 | Multi-scope (frame/window/session); warm-up conditions per-window | Multi-
scale/hierarchical modeling + play-break + phase recognition | Tjondronegoro ICME 2003; GL-MSTCN
(10.1186/s12938-022-01048-w); MS-TCN (arXiv:1903.01945) | partial-overlap |
583
584 ****In-domain prior art that already hard-codes our cues**** (so our delta is the **no-branch*
discipline*, not the
585 cues): non-broadcast badminton rally start/end (See et al. 2024/2025), shuttle-state/hit
detection (MonoTrack
586 arXiv:2204.01899; Hsu/Yu/Cheng, Sensors 2024 24(13):4372), service-fault (Goh et al.,
Sensors 2023 23(24):9759),
587 tennis in-play state machines (10.1080/19346182.2013.819007), badminton point-
segmentation (Ghosh et al. 2018,

588 arXiv:1712.08714), ShuttleSet (arXiv:2306.04948).
 589
 590 **### 13.2 Terminology to adopt (gloss our nouns with the field's on first use)**
 591
 592 | Our term | Field's established term | Anchor |
 593 |---|---|---|
 594 | `RallyCue` / "producer" | ****labeling function**** / weak-supervision source; ****base**
model** in a stack | Snorkel; Wolpert 1992 |
 595 | "Signal not verdict / no special-casing" | ****late / score-level fusion****; arbiter =
****level-1 combiner / meta-learner**** | Snoek 2005; Wolpert 1992 |
 596 | ADR-004 LR scorer (`backend/eval/classifier.py`) | ****stacked meta-learner / level-1**
combiner** | Wolpert 1992 |
 597 | "Detection?decision / persist once, re-score" | ****frozen-backbone cached features****;
****decoupled proposal/classification****; offline re-scoring | S-CNN; Decouple-SSAD |
 598 | `WindowAnalyzer` (window-scope) | ****temporal action proposal**** features /
****actionness**** | S-CNN; BMN |
 599 | Session-scope / warm-up | ****video-level temporal context / phase segmentation****;
****play-break**** outer layer; ****global-local**** | Tjondronegoro 2003; GL-MSTCN |
 600 | `ConsistencyRule` reconciler | ****model/consistency assertions****; ****constraint-based**
repair**; ****boundary refinement****; ****activity grammar / constrained decode**** | Kang 2020;
 HoloClean; ASRF; Activity Grammars |
 601 | "Report-first linter" | ****data validation / schema anomalies**** ("unit tests for data")
 | TFDV; DeeQu |
 602 | "Over-merged / split" . "tighten boundary slack" | ****over-segmentation error**** .
****boundary regression/refinement**** | MS-TCN; ASRF; CBR |
 603 | "LOVO, group by match" | ****leave-one-group-out / -subject-out (LOGO/LOSO)**** grouped CV
 | sklearn `GroupKFold`; Austin 2025 |
 604 | Temporal-IoU metric | ****mAP@tIoU**** (TAL) + ****F1@{10,25,50} & edit score**** (TAS) +
****tight average-mAP**** (spotting) | Lea 2017; SoccerNet; Soares 2022 |
 605
 606 Keep the internal nouns; just gloss each on first use. Low-effort, high-leverage ?
 reviewers map instantly to
 607 30 years of literature and see we know the failure modes.
 608
 609 **### 13.3 What we can / cannot claim**
 610
 611 - ? ****The integrated governance envelope as a whole**** ? no found paper bundles all of:
 report-first
 612 contradiction detection over a ***persisted temporal decision*** + auditable structured
 interval corrections
 613 (truncate/split/drop/tighten) + ****per-rule A/B lift-gating before auto-apply**** +
 idempotency/precedence +
 614 human-in-the-loop + feeding a learned (value, confidence) scorer.
 615 - ? ****"Signal not verdict" as a deliberate *design-philosophy delta***** vs. the in-domain
 prior art that
 616 hard-codes the same cues as if/else verdicts (state it as a philosophy delta, not a
 capability they lack).
 617 - ? ****Warm-up/practice as a soft session-scope feature**** ? thin prior art (only
 patents/coaching content
 618 surfaced); claim cautiously, validate from scratch.
 619 - ? The ****fusion mechanism**** (multi-cue features ? light scorer for rally segmentation)
 ? ~20 yrs old in
 620 racket sports (CVIU 2009; Ghosh et al. 2018 learn an SVM over per-frame features ?
 smoothed rallies).
 621 - ? The ****persist-once/re-score substrate**** as novel ? it is the ***default*** of the TAD
 field (datasets ship
 622 precomputed features so cheap heads re-train). Claim only the ***idempotent/versioned**
discipline*.
 623 - ? The ****cross-check-and-correct insight**** ? Hsu et al. 2024 already cross-check
 shuttle-vs-swing streams;
 624 EventAnchor already does human-in-the-loop suggest-then-verify. Our delta is
***measured-before-apply** +
 625 auditability*, not the cross-check.
 626 - ? ****Static single-camera as a novel setting****, and ? the ****no-regression/flag-gated**
harness** as a

```

627     primitive (textbook MLOps) ? claim only "this discipline on a deliberately tiny golden
set."
628
629     ### 13.4 Course corrections (owner-gated; prioritized)
630
631     **Tier 1 ? protect the honesty of "measured lift" on ~6 matches (do first):**
632     - **C1 ? bias-aware LOVO.** Standard leave-one-out on ~6 imbalanced groups induces a
train-vs-heldout label
633     anti-correlation that biases the estimate (a LR scored ~0.23 auROC on *random* data in
the cited study).
634     Use class-balance-preserving folds (RL00CV) and **report per-fold variance + CIs,
never a single mean**, so
635     the promotion gate can't promote noise. *(Austin/Pe'er/Korem 2025, arXiv:2406.01652.)*
? **harness/eval.**
636     - **C2 ? calibrate each producer's confidence** (Platt/isotonic) *before* the LR, so a
CNN softmax, an audio
637     onset, and an integer net-crossing count are comparable and the over-confident cue
can't swamp the arbiter.
638     *(MoCaE arXiv:2309.14976; modality competition arXiv:2203.12221.)* ? **producers +
scorer.**
639     - **C3 ? out-of-fold (honest) stacking.** When a producer is itself learned (shuttle-
state), feed the LR its
640     *out-of-fold* outputs; extend the group-by-match split to producer training. *(Wolpert
1992.)* ?
641     **detection?decision + harness.**
642     - **C4 ? report the field's metrics.** Add **segmental F1@{10,25,50} + edit score** and
a **tight/loose
643     mAP@? sweep alongside temporal-IoU: tIoU is nearly blind to over-segmentation, so a
reconciler win (or an
644     IoU-chasing rule that shreds structure) is invisible under tIoU alone. *(MS-TCN; ASRF;
SoccerNet
645     arXiv:1804.04527; Soares 2022 arXiv:2206.07846.)* ? **harness/eval + reconciler.**
646
647     **Tier 2 ? architecture/discipline upgrades:**
648     - **C5 ? resolve reconciler rules GLOBALLY** (an activity-grammar / constraint-aware
Viterbi decode), not as
649     stacked local edits with fixed precedence (which can conflict / be non-idempotent
across orderings). Model
650     the structural priors (transition/duration) from data. *(Activity Grammars
arXiv:2312.04266; Constraint-Aware
651     Decoding arXiv:2605.10149 ? read before citing.)* ? **reconciler.**
652     - **C6 ? brand the reconciler as "model/consistency assertions,"** attach a per-
correction confidence, and
653     route only high-confidence corrections to auto-apply, the rest to a margin-ranked
human queue; version the
654     rule set as a TFDV-style owner-ratified anomaly schema. *(Kang 2020; HoloClean ~90%
repair precision ? ~10%
655     bad; Confident Learning arXiv:1911.00068.)* ? **reconciler + harness.**
656     - **C7 ? a few EARLY-fusion interaction columns** (e.g. `shuttle_in_air AND
person_two_sided`) ? a linear
657     arbiter can't represent AND/XOR; hand-crossed products recover it without leaving
"signal not verdict."
658     *(Snoek 2005.)* ? **producers + scorer.**
659     - **C8 ? represent a missing cue as an explicit `is_present`/uncertainty column, not a
silent 0** ? "cue
660     absent" ? "cue says no" for a linear arbiter. *(DBF arXiv:1511.03183.)* ? **producers
+ scorer.**
661     - **C9 ? keep warm-up/session-state a SOFT feature, never a hard pre-filter**, and add a
court/context
662     invariance probe: the static court co-occurs with rallies, so the scorer can learn
"court visible" instead
663     of "point in play" (action-context confusion). Evaluate on warm-up stretches that
share the court but lack
664     point cadence. *(arXiv:2103.16155; GL-MSTCN.)* ? **session-scope producers + scorer.**
665

```

```

666     **Tier 3 ? leverage external resources / higher-effort:**
667     - **C10 ? model cue CORRELATIONS** (Snorkel/MeTaL label-model baseline; strong L1/L2):
net-crossing,
668     two-sidedness, shuttle-in-air co-fire, so an unregularized LR double-counts the
bundle. *(arXiv:1711.10160.)*
669     - **C11 ? a temporal-smoothing pass** (Kalman/HMM/Viterbi over the per-window posterior,
or an MS-TCN
670     truncated-MSE anti-fragmentation penalty) before cutting intervals ? SmartTennisTV
hits F1 ~97.5% largely
671     via Kalman-smoothing noisy per-frame predictions. *(arXiv:1801.01430; MS-TCN.)* ?
**reconciler/scorer.**
672     - **C12 ? pull ShuttleSet/ShuttleSet22 + CoachAI** as an external benchmark + weak-
supervision corpus to
673     attack the ~6-match bottleneck. *("Less is More" arXiv:1903.00859; ShuttleSet
arXiv:2306.04948.)* ?
674     **harness/eval + producers.**
675     - **C13 ? benchmark the rule reconciler against a learned boundary refiner** (ASRF/CBR)
to *justify* staying
676     rule-based, and ground rules in MonoTrack-style physics/cadence as **soft features,
not hard rules** (hard
677     cadence rules would re-introduce special-casing and break on doubles/lets/warm-up).
*(MonoTrack
678     arXiv:2204.01899; CBR arXiv:1705.01180.)* ? **reconciler.**
679
680     ### 13.5 Pitfalls we are actively walking into (risk register)
681
682     - **Over-segmentation hidden by tIoU** ? the dominant error mode of cheap segment
decisions; a per-window
683     scorer can look good while fragmenting rallies. ? C4.
684     - **Late-fusion miscalibration / dominant-cue swamping** (over-confident shuttle CNN,
low-discrimination
685     audio). ? C2.
686     - **L00CV distributional bias + tiny-N variance** ? promotion gate promotes noise
without CIs. ? C1.
687     - **Stacking leakage** (in-sample learned-producer outputs) ? C3; and **threshold-
tuning-on-test** ? keep
688     *every* tunable knob (windowing, merge/NMS, regularization, reconciler thresholds)
*inside* the LOVO loop.
689     - **Proposal-recall ceiling** ? the decision can't recover a rally the windowing missed;
measure window
690     recall (AR@AN) separately or you'll misattribute recall loss to the scorer.
691     - **Report-first decaying into silent rewrite** ? without per-rule lift gating + a
confidence gate +
692     idempotency tests, an over-eager rule systematically truncates correct long rallies.
693     - **Static-tripod transfer gap** ? broadcast cues (camera-switch, replays, crowd audio)
that drive the
694     tennis/soccer baselines *don't exist* on our footage; don't assume their F1s transfer.
695     - **Detector domain gap inherited downstream** ? WASB/TrackNet/MonoTrack are trained on
other rigs;
696     per-frame shuttle accuracy degrades off-distribution and every cue inherits that
error. **Quantify detector
697     accuracy on our footage before trusting any cue column** (currently unmeasured).
698
699     ### 13.6 Residual gaps & honesty notes
700
701     - **Fabrications: none** across ~40 citation checks. Corrected author-lines applied
above: CVIU 2009 =
702     **Liu, Huang, Jiang, Xing, Ye, Gao** (not "Kolekar"); SmartTennisTV = **Ghosh &
Jawahar**; Sensors 2024
703     shuttle-fusion = **Hsu, Yu, Cheng**. Metadata fixes: L00CV-bias is **Science
Advances** (10.1126/sciadv.adx6976);
704     ShuttleSet (KDD) = arXiv:2306.04948 vs ShuttleSet22 = arXiv:2306.15664 (distinct); SF-
Net evaluates on
705     THUMOS14/GTEA/BE0ID.
706     - **Read before citing** (verified to *exist*, not deep-read; both 2026, concurrent ?

```

don't void novelty but
707 are the nearest precedents): Constraint-Aware Decoding (arXiv:2605.10149, the
reconciler end-state analog)
708 and TennisExpert (arXiv:2603.13397, an audio-hits?cluster?filter rally pipeline). PCL-
Former
709 (arXiv:2507.06411) ? authorship unverified; cite without asserting authors.
710 - ****Weakest-evidence claims (state plainly):**** warm-up-as-soft-feature (novel by
absence of counterexample,
711 the high from-scratch-validation risk) and the measured-before-apply reconciler bundle
(also absence-based).
712 - ****Not yet searched:**** a recent badminton full-match frame-level rally/interruption
dataset (would relieve
713 scarcity); doubles-specific cue behavior; calibration for *count-valued* cues (net-
crossings); whether
714 RL00CV interacts badly with grouped folds at N=6 (the cited study is record-wise, not
group-wise).
715

12. user (2026-06-19T15:46:49.191Z)

```

1      # How rally detection works (the 2-layer mental model)
2
3      A short conceptual map of how we go from raw video to rally `[start, end]` segments ?
4      and, crucially, **which layer our annotation can improve.**
5
6      ```
7      video ??? frames ??? [DETECTION] ??? shuttle (x,y,visible) per frame
8              ?
9      (WASB CNN,          per-frame trajectory
10     frozen weights)      ?
11                          ?
12     velocity signal  (how fast the shuttle moves)
13                          ?
14     [SEGMENTATION]  ??? the 3 knobs we tune
15                          ?
16                          ?
17     rally windows  [start, end] ??? IoU vs golden
labels
18     ```
19
20     ## Two layers ? keep them separate in your head
21
22     **1. DETECTION ? "where is the shuttle, this frame?"**
23     WASB is a CNN that, for each frame, outputs a heatmap ? the shuttle's `(x, y)` + a
24     visibility/confidence. Its weights were learned (by gradient descent, on shuttle-
coordinate
25     labels) and we use them **frozen**. We chose a *shuttle* detector on purpose: "shuttle
in
26     flight" is the signal for "a rally is happening," and it's **camera-agnostic** ? unlike
27     player-motion, which fooled the old heuristic on broadcast cameras that pan with the
players.
28     (Your videos are static tripod, so player-motion would also work ? but shuttle-motion is
the
29     robust general bet.)
30
31     **2. SEGMENTATION ? "when is a rally?"**
32     From the per-frame `(x, y)` we compute the shuttle's **velocity** (displacement between
frames).
33     Then three knobs turn that into rally intervals:
34     - **`velocity_thresh`** ? a frame is "active" only if the shuttle is moving faster than
this
35     (in flight, not lying on the floor / being picked up).
36     - **`merge_gap`** ? active frames within this many seconds get joined into one rally
(bridges
37     brief gaps: a shuttle momentarily occluded, or the beat between two hits).
```

```

38     - min_window_duration ? throw away windows shorter than this (blips / false
flickers).
39
40     Output: a list of [start, end] windows = the rallies.
41
42     ## Where the annotation fits (the key point)
43     Our Roora/golden labels are rally [start, end] times ? they say when rallies
are, not
44     where the shuttle is*. So they can't teach WASB's detector (that needs (x,y) labels).
They
45     can grade and tune the segmentation layer: we score predicted windows against
the golden
46     rallies with temporal IoU (how much they overlap), and pick the 3 knobs that
maximise it ?
47     calibrate on some rallies, measure on held-out ones so the number is honest.
48
49     ## Where sparks could land (open questions ? not commitments)
50     Things worth a shower-thought; any of these could beat 3 hand-tuned thresholds:
51     - Richer features than raw velocity: acceleration, trajectory continuity (a real
rally is a
52     smooth arc that survives occlusion), shuttle height/arc, or direction reversals
(each
53     reversal ? a shot ? ties to shots_count!).
54     - A tiny learned segmenter instead of 3 thresholds: feed the trajectory features to
a small
55     model trained on the golden rally labels (still the segmentation layer, just smarter).
56     - Hysteresis (two thresholds: enter-rally high, stay-in-rally low) ? robust to mid-
rally dips.
57     - Rhythm/periodicity: rallies have a hit cadence; dead time doesn't.
58     - Fusion: combine shuttle-motion (WASB) with player-motion for static cams. ?
designed in
59     [MULTI_SIGNAL_FUSION_PLAN.md] (one rally-fusion layer;
the person cue is
60     ADR-007's parked #2 and rally_gate.py's named-but-unbuilt "Gate B").
61     - Per-video calibration as a feature: "watch a few labeled rallies ? auto-tune to
this
62     court/lighting ? segment the rest sharply" ? exactly your earlier idea, at the right
layer.
63
64     The whole point of per-video calibration: a handful of your labeled rallies nudges
these knobs
65     to your court/camera, and we report the IoU lift on the rallies it never saw.
66

```

13. user (2026-06-19T15:46:49.715Z)

```

1     # Ablation layer (Phase 0) ? per-signal marginal contribution
2
3     A systematic, repeatable version of the earlier one-off ablation:
4     backend/eval/ablation.py runs it and it EXCLUDES one signal, measures whether quality
dropped,
5     and reports each signal's marginal F1 (full set vs full-minus-that-signal) with a
bootstrap
6     95% CI + paired sign test + the C4 over-segmentation ratio. The broader design (the
ledger time
7     series, interaction handling, per-user ablation, the phased plan) is not yet split into
its own
8     doc; this note covers only the shipped Phase-0 runner.
9
10    ## What it is (and is not)
11
12    A pure driver. No new scoring/stats math ? every number defers to the shipped
13    experiment.compare + eval_stats engine, exactly as compare was a driver over
14    metrics/calibration. For a signal s:
15

```

```

16     ``
17     ?(s) = score(full set S) ? score(S \ {s})          # leave-one-out drop, ?F1 primary
18     ``
19
20     is `experiment.compare(videos, variant_a=without_s, variant_b=full)` ? A is the ablated
(smaller)
21     variant, B is full, so the reported **B?A** ?F1 IS ?(s): a per-video-paired ?F1 with
bootstrap CI
22     + exact sign test, LOVO-honest for learned variants.
23
24     A signal is either a **feature column** (person/audio/shuttle columns the learned scorer
weights,
25     ablated by dropping the column) or a **gate knob** (Gate-A's `min_crossings` threshold,
ablated by
26     zeroing the knob). The runner handles both uniformly.
27
28     ## Two granularities
29
30     - **LOSO** (`--granularity signal`) ? leave one column-set out at a time (fine-grained).
31     - **LOGO** (`--granularity group`) ? leave a whole producer's columns out at once (the
owner's
32     "is this old SIGNAL still relevant?" question at cue granularity). Default.
33
34     ## The honest small-N contract
35
36     Inherited from `eval_stats` / `per_user_gate` (no new statistics):
37
38     - **Never a bare mean** ? every row ships a CI + the exact sign test.
39     - A signal **earns its keep** only when removal drops F1 with the ?F1 **CI excluding 0
AND the
40     sign test agreeing** (the `per_user_gate` bar, in reverse).
41     - **Structural guards are exempt from auto-demotion.** Gate-A (the held-shuttle false-
positive
42     killer) is marked `structural`; it is reported honestly but verdict is always
43     `structural-protected`, never flagged dead weight on "no measured lift" ? its value is
in
44     failure modes the golden corpus may not contain.
45     - The verdict vocabulary is closed and never over-claims: `clears-bar` / `suggestive` /
`inert`
46     / `structural-protected`. One wash at n=6 is `inert` (a candidate), never
"irrelevant".
47
48     ## Run it
49
50     ```bash
51     # LOGO contribution table over the learned full set (shuttle+gate_a+person+audio):
52     python -m backend.eval.ablation --features-dir output [--granularity signal] \
53         [--pdf ablation.pdf] [--json ablation.json]
54
55     # Reproduce the hand-measured Gate-A result (the framework's litmus):
56     python -m backend.eval.ablation --features-dir output --litmus
57     ```
58
59     The corpus is the 6 persisted golden feature JSONs (`output/*_golden_features.json`,
written by
60     `fusion_golden.py` ? persist once on the GPU, re-score forever offline). `--pdf` writes
a clean
61     reportlab report (table of marginal contribution + CI + sign test + a verdict column).
62
63     ## Litmus (regression guard on the math)
64
65     `tests/test_ablation.py::test_litmus_reproduces_gate_a` asserts that, on the 6 real
golden JSONs,
66     the runner reproduces the recorded `QUALITY_ITERATIONS` Gate-A result within tolerance:
67

```

```

68 | metric | recorded (2026-06-14) | reproduced |
69 |---|---|---|
70 | marginal ?F1 | +0.074 | +0.074 |
71 | sign test | 6W/0L, p=0.031 | 6W/0L, p=0.031 |
72 | 95% CI | [+0.042, +0.104] | [+0.042, +0.105] |
73 | over-seg ratio | 1.68 ? 1.01 | 1.68 ? 1.01 |
74
75 The test skips cleanly when the golden substrate is absent (CI) and runs when present
(owner box).
76 The rest of the suite runs GPU-free on deterministic synthetic candidate rows.
77

```

14. assistant (2026-06-19T15:46:53.623Z)

Now I have a strong picture of the code. Let me verify the key claims by examining the eval/experiment/windowing/ablation code and checking a few specific assertions in the docs, particularly around the corpus size (n=6 vs n=13), the `experiment.predict` feature columns, and the ablation litmus.

15. user (2026-06-19T15:46:55.045Z)

```

200 features_dir: str,
201 *,
202 shuttle_columns: Sequence[str],
203 person_columns: Sequence[str],
204 audio_columns: Sequence[str] = (),
205 audio_policy: str = "omit",
206 skip_logger: logging.Logger = logger,
207 skip_message: str,
208 no_audio_error: str = "{path} has no audio features",
209 ) -> List["VideoCandidates"]:
210     """Shared golden-feature corpus loader behind the three structurally-identical
loaders
211     (`ablation.load_videos` / `fusion_compare.load_videos` /
`fusion_audio.load_videos_with_audio`).
212
213     Reads every FULL-FUSION-SCHEMA ``*_golden_features.json`` in ``features_dir`` into a
214     ``VideoCandidates`` (intervals + the caller-supplied feature columns + golden gts).
Leaner
215     velocity-windowing-only files (no ``shuttle``/``person`` cue dicts ? the schema the
216     corpus-growth path persists) are SKIPPED, not crashed on (see
``_is_fusion_schema``), so a
217     MIXED corpus loads the fusion subset cleanly.
218
219     The column lists are ALWAYS supplied by the caller ? this loader never hardcodes
columns. The
220     three callers differ ONLY in:
221
222     - ``audio_policy`` ? how the audio block is handled, checked BEFORE reading any
audio column
223     so a windowing-only file is skipped before either branch applies:
224     * ``"omit"`` ? no audio columns are loaded (``audio_columns``
ignored);
225     * ``"tolerate-missing"`` ? audio columns load, a missing block defaults them
to 0.0;
226     * ``"require"`` ? a fusion file lacking audio raises ``SystemExit``
(via
227     ``no_audio_error`` formatted with ``path``), mirroring "run ---augment``
first".
228     - ``skip_logger`` / ``skip_message`` ? the per-caller logger + ``%s``-templated
info line
229     emitted when a windowing-only file is skipped (kept byte-identical per caller).
230     """
231     if audio_policy not in ("omit", "tolerate-missing", "require"):
232         raise ValueError(

```

```

233         f"audio_policy must be 'omit'/'tolerate-missing'/'require', got
{audio_policy!r}")
234     videos: List[VideoCandidates] = []
235     for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
236         with open(path, encoding="utf-8") as f:
237             d = json.load(f)
238             cands = d["candidates"]
239             if not _is_fusion_schema(cands):
240                 skip_logger.info(skip_message, os.path.basename(path))
241                 continue
242             if audio_policy == "require" and "audio" not in cands[0]:
243                 raise SystemExit(no_audio_error.format(path=path))
244             intervals = [(float(c["start"]), float(c["end"])) for c in cands]
245             features: Dict[str, List[float]] = {}
246             for fn in shuttle_columns:
247                 features[fn] = [float(c["shuttle"][fn]) for c in cands]
248             for fn in person_columns:
249                 features[fn] = [float(c["person"][fn]) for c in cands]
250             if audio_policy == "tolerate-missing":
251                 for fn in audio_columns:
252                     features[fn] = [float((c.get("audio") or {}).get(fn, 0.0)) for c in
cands]
253             elif audio_policy == "require":
254                 for fn in audio_columns:
255                     features[fn] = [float(c["audio"][fn]) for c in cands]
256             gts = [(float(a), float(b)) for a, b in d["gts"]]
257             videos.append(VideoCandidates(name=d["name"], intervals=intervals,
258                                         features=features, gts=gts))
259     return videos

```

16. user (2026-06-19T15:46:55.504Z)

```

1     """Single-sourced candidate-windowing presets ? the eval?serve foundation fix.
2
3     **The bug this closes.** The offline eval substrate (`distill_local.build_candidates`
?
4     ``fusion_golden`` ? the golden feature JSONs the ablation/compare path re-scores) was
built
5     on a *recall-oriented PROPOSAL* windowing (`vt=0.005, merge_gap=1.0, min_window=0.5`):
6     deliberately permissive so a learned scorer/gate can filter the junk. But the SERVED
pipeline
7     (`TrajectoryHybridSegmenter.process_video`) windows COARSELY (`vt=0.02,
merge_gap=2.0,
8     min_window=2.0` ? the `config.indexing` defaults). A cue measured on the proposal set
is
9     measured on candidates **production never serves**, so a "win" need not transfer. This
is
10    exactly how Gate-A scored **+0.074** on the proposal set yet **?0.008** on the served
set
11    (`scratch/GATE_AB_NUMBERS_AND_IMPLICATIONS.md` vs `output/gateA_served_verify/`).
12
13    **The fix.** One module owns the two named windowings as :class:`WindowingPreset`s, and
the
14    SERVED preset is **single-sourced from the same Pydantic config defaults production
reads**
15    (:class:`backend.config.models.IndexingConfig`) ? eval and serve can no longer drift.
Helpers
16    build candidates under either preset, and :func:`served_windowing_from_config` lifts the
17    *actual* served windowing out of a live config (overrides included), so an A/B can
target the
18    operating point a given deployment truly serves.
19
20    The companion rule ? *a cue is promotable only if it does NOT regress at the SERVED
21    windowing* ? lives in :mod:`backend.eval.promotion` (which composes ``eval_stats`` +
22    ``per_user_gate``). This module is the **what windowing** ; that one is the **promotion

```

```

gate**.
23
24     Pure, offline, stdlib + config only. Nothing here changes existing eval behaviour until
a
25     caller opts in (``distill_local`` re-points its module-level presets here, byte-
identically).
26     """
27     from __future__ import annotations
28
29     from dataclasses import dataclass
30     from typing import Any, Dict, List, Mapping, Tuple
31
32     from backend.config.models import IndexingConfig
33
34     Interval = Tuple[float, float]
35
36     __all__ = [
37         "WindowingPreset",
38         "SERVED",
39         "PROPOSAL",
40         "PRESETS",
41         "served_windowing_from_config",
42         "build_windows",
43         "windows_to_intervals",
44     ]
45
46
47     @dataclass(frozen=True)
48     class WindowingPreset:
49         """A named (velocity_thresh, merge_gap, min_window_duration) windowing operating
point.
50
51         ``velocity_thresh`` ? normalised per-second shuttle velocity above which a frame
counts as
52         "in flight"; ``merge_gap`` ? seconds within which active frames coalesce into one
window;
53         ``min_window_duration`` ? windows shorter than this are dropped. These are exactly
the three
54         knobs ``TrackNetRunner.trajectory_to_action_windows`` takes, so a preset *is* the
windowing.
55         """
56
57         name: str
58         velocity_thresh: float
59         merge_gap: float
60         min_window_duration: float
61         #: 0 = OFF (default). When > 0, windows longer than this are split at their largest
internal
62         #: inactivity gap ? the bounded-windowing over-merge fix (W1,
RALLY_QUALITY_RESEARCH.md §6).
63         max_window_duration: float = 0.0
64         #: When True, ``max_window_duration`` is a HARD cap ? an over-long window with no
internal
65         #: inactivity gap is chopped at its midpoint until ? the cap (continuously-active
fix). OFF by default.
66         hard_cap: bool = False
67         #: R4 rally-END inactivity trim (s, 0 = OFF): trim a window's post-rally slow-drift
tail back to
68         #: the last frame whose trailing window stays dense with FAST motion. The measured
rally-end fix.
69         inactivity_end: float = 0.0
70         #: velocity above which motion counts as "rally" (vs slow drift) for the R4 trim;
min fast-fraction.
71         inactivity_rally_thresh: float = 0.08
72         inactivity_min_density: float = 0.34

```

```

73      #: R5 blob-fallback (default OFF). After the cap split + R4 trim, midpoint-chop any
group still
74      #: longer than ``blob_factor`` × ``max_window_duration`` (the gap-less, R4-survived
blob ?
75      #: mahadevpura-1's ~11× residual) and flag it. Runs AFTER R4 (vs ``hard_cap``
before) and the
76      #: factor keeps it surgical (a normal long rally is left alone).
77      blob_fallback: bool = False
78      blob_factor: float = 4.0
79
80      def as_tuple(self) -> Tuple[float, float, float]:
81          """(velocity_thresh, merge_gap, min_window_duration)`` ? the legacy tuple
shape the
82          ``trajectory_to_action_windows`` / ``distill_local`` call sites already speak.
(The
83          bounded-windowing knob ``max_window_duration`` is passed separately by
``build_windows``.)"""
84          return (self.velocity_thresh, self.merge_gap, self.min_window_duration)
85
86      def to_dict(self) -> Dict[str, Any]:
87          return {
88              "name": self.name,
89              "velocity_thresh": self.velocity_thresh,
90              "merge_gap": self.merge_gap,
91              "min_window_duration": self.min_window_duration,
92              "max_window_duration": self.max_window_duration,
93              "hard_cap": self.hard_cap,
94              "inactivity_end": self.inactivity_end,
95              "inactivity_rally_thresh": self.inactivity_rally_thresh,
96              "inactivity_min_density": self.inactivity_min_density,
97              "blob_fallback": self.blob_fallback,
98              "blob_factor": self.blob_factor,
99          }
100
101
102      def _served_preset_from_defaults() -> WindowingPreset:
103          """The SERVED windowing, lifted from the SAME Pydantic config defaults production
reads.
104
105          ``TrajectoryHybridSegmenter.process_video`` reads exactly these three:
106          - velocity: ``idx_cfg[<family>].velocity_threshold`` (default 0.02 ?
wasb/fusion/tracknet)
107          - merge_gap: ``idx_cfg.dead_time_merge_gap`` (default 2.0)
108          - min window: ``idx_cfg.min_action_window_duration`` (default 2.0)
109
110          Sourcing them from :class:`IndexingConfig`'s defaults (not a hand-typed tuple) is
the whole
111          point: change a served default in ``config/models.py`` and the eval SERVED preset
follows,
112          so the two can never silently diverge again.
113          """
114          idx = IndexingConfig() # construct with defaults ? the served operating point
115          return WindowingPreset(
116              name="served",
117              velocity_thresh=float(idx.wasb.velocity_threshold), # == fusion/tracknet
default 0.02
118              merge_gap=float(idx.dead_time_merge_gap),
119              min_window_duration=float(idx.min_action_window_duration),
120              max_window_duration=float(idx.max_window_duration), # W1 bounded-windowing
(default 0=OFF)
121              hard_cap=bool(idx.window_hard_cap), # W1 hard-cap fallback
(default OFF)
122              inactivity_end=float(idx.window_inactivity_end), # R4 rally-end trim
(default 0=OFF)
123              inactivity_rally_thresh=float(idx.inactivity_rally_thresh),

```

```

124         inactivity_min_density=float(idx.inactivity_min_density),
125         blob_fallback=bool(idx.window_blob_fallback),          # R5 blob-fallback
(default OFF)
126         blob_factor=float(idx.window_blob_factor),
127     )
128
129
130     #: The COARSE windowing production actually serves (config defaults; ~0.02/2.0/2.0). A
cue is
131     #: only promotable if it holds up HERE ? the operating point real users get.
132     SERVED: WindowingPreset = _served_preset_from_defaults()
133
134     #: The recall-oriented PROPOSAL windowing the offline substrate is built on
(deliberately
135     #: permissive: propose many, let the classifier/gate filter). NOT what production serves
? a
136     #: win here must be re-confirmed on SERVED before promotion. Mirrors the historical
137     #: ``distill_local.PROPOSAL`` tuple (0.005/1.0/0.5) so the existing golden JSONs stay
valid.
138     PROPOSAL: WindowingPreset = WindowingPreset(
139         name="proposal", velocity_thresh=0.005, merge_gap=1.0, min_window_duration=0.5
140     )
141
142     #: Name ? preset, so a CLI / config string ("served" | "proposal") selects one.
143     PRESETS: Dict[str, WindowingPreset] = {SERVED.name: SERVED, PROPOSAL.name: PROPOSAL}
144
145
146     def served_windowing_from_config(config: Any, family: str = "wasb") -> WindowingPreset:
147         """The SERVED windowing a *specific* live config serves (overrides honoured).
148
149         Reproduces ``TrajectoryHybridSegmenter.process_video``'s knob reads exactly, against
an
150         arbitrary config (the typed :class:`AppConfig` or a plain dict) and segmenter
``family``
151         (``"wasb"`` / ``"fusion"`` / ``"tracknet"``). Use this when a deployment overrode a
served
152         default in ``config.json`` ? the module-level :data:`SERVED` is the *default*
operating point;
153         this is *this config's* operating point. Falls back to the served defaults for any
absent key.
154         """
155         idx = _as_mapping(config, "indexing")
156         family_cfg = _as_mapping(idx, family)
157         return WindowingPreset(
158             name=f"served:{family}",
159             velocity_thresh=float(family_cfg.get("velocity_threshold",
SERVED.velocity_thresh)),
160             merge_gap=float(idx.get("dead_time_merge_gap", SERVED.merge_gap)),
161             min_window_duration=float(idx.get("min_action_window_duration",
SERVED.min_window_duration)),
162             max_window_duration=float(idx.get("max_window_duration",
SERVED.max_window_duration)),
163             hard_cap=bool(idx.get("window_hard_cap", SERVED.hard_cap)),
164             inactivity_end=float(idx.get("window_inactivity_end", SERVED.inactivity_end)),
165             inactivity_rally_thresh=float(idx.get("inactivity_rally_thresh",
SERVED.inactivity_rally_thresh)),
166             inactivity_min_density=float(idx.get("inactivity_min_density",
SERVED.inactivity_min_density)),
167             blob_fallback=bool(idx.get("window_blob_fallback", SERVED.blob_fallback)),
168             blob_factor=float(idx.get("window_blob_factor", SERVED.blob_factor)),
169         )
170
171
172     def _as_mapping(obj: Any, key: str) -> Mapping[str, Any]:
173         """``obj[key]`` as a plain mapping, whether ``obj`` is a dict or a typed config

```

```

section
174     (``AppConfig.get`` returns nested sections as dicts). Empty mapping if absent/non-
mapping. """
175     if obj is None:
176         return {}
177     if isinstance(obj, Mapping):
178         sub = obj.get(key, {})
179     elif hasattr(obj, "get"):
180         sub = obj.get(key, {})
181     else:
182         sub = getattr(obj, key, {})
183     return sub if isinstance(sub, Mapping) else {}
184
185
186     def build_windows(points: List[Any], fps: float, frame_width: float,
187                       preset: WindowingPreset) -> List[Dict[str, Any]]:
188         """Trajectory ``points`` ? candidate windows under ``preset`` (the one windowing
call site).
189
190         A thin wrapper over ``TrackNetRunner.trajectory_to_action_windows`` that takes a
named preset
191         instead of three loose floats, so eval candidate-building and the served path
reference the
192         SAME knobs by name. Imported lazily to keep this module import-light (no detector
deps). """
193         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
194
195         vt, mg, mwd = preset.as_tuple()
196         return TrackNetRunner.trajectory_to_action_windows(
197             points, fps=fps, frame_width=frame_width,
198             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
199             max_window_duration=preset.max_window_duration,
200             window_hard_cap=preset.hard_cap,
201             window_inactivity_end=preset.inactivity_end,
202             inactivity_rally_thresh=preset.inactivity_rally_thresh,
203             inactivity_min_density=preset.inactivity_min_density,
204             window_blob_fallback=preset.blob_fallback,
205             window_blob_factor=preset.blob_factor)
206
207
208     def windows_to_intervals(windows: List[Dict[str, Any]]) -> List[Interval]:
209         """``[{start, end, ...}]`` ? ``[(start, end)]`` ? the interval shape the scorer/gate
speak. """
210         return [(w["start"], w["end"]) for w in windows]
211

```

17. user (2026-06-19T15:46:56.329Z)

```

240         }
241
242         windows = []
243         group = [active_frames[0]]
244         for af in active_frames[1:]:
245             if af[0] - group[-1][0] <= max_gap_frames:
246                 group.append(af)
247             else:
248                 windows.append(_finalize(group))
249                 group = [af]
250         windows.append(_finalize(group))
251
252         # Filter out extremely short action windows
253         min_window_duration = self.config.get("indexing",
{}).get("min_action_window_duration", 2.0)
254         windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
255

```

```

256         return windows
257
258     def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
259                             frame_skip: int = 3) -> Dict[str, Any]:
260         """Per-frame person detections over ``[start_frame, end_frame]`` of the ORIGINAL
261         video.
262         Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
263         ``extract_action_windows_from_chunk`` (which detects ITS OWN windows and
264         discards the
265         per-frame boxes), this returns the raw per-frame detections so the fusion
266         features can
267         align them with the shuttle trajectory. It reads the source by absolute frame
268         index
269         (seek + sequential read), NOT a re-extracted clip, so the returned frame indices
270         match
271         the WASB trajectory's ``.frame`` exactly ? that alignment is what
272         ``fusion_features.shuttle_player_colocation`` / ``two_sided_motion`` rely on.
273         Returns ``{"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width",
274         "frame_height"}``. A fresh window-local ByteTrack tracker is used. ? cv2 frame-
275         seek
276         accuracy varies by codec (it lands near the requested keyframe then decodes
277         forward);
278         treat the indices as  $\pm$  a few frames on real footage ? fine for window-level
279         features.
280         """
281         self.lazy_load_model()
282         cap = cv2.VideoCapture(video_path)
283         if not cap.isOpened():
284             logger.error(f"Could not open {video_path}")
285         return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}

```

18. assistant (2026-06-19T15:47:01.773Z)

The seed claim is confirmed: `shuttle\_player\_colocation` is BUILT in `fusion\_features.py:168` and in `PERSON\_FEATURE\_NAMES`, wired in `fusion\_hybrid.py` under `cue\_flags.person`, default-OFF and mock-tested. Now let me check the `experiment.predict` feature columns claim (ANALYZERS doc says `predict()` treats `players\_in\_court` as a feature column at line 213), and verify the ablation litmus corpus size (n=6 vs n=13 question). Let me look at experiment.py around 213 and the ablation.py litmus.

19. user (2026-06-19T15:47:03.198Z)

```

1     """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3     This is the thin variant/experiment layer that `docs/EXPERIMENT_HARNESS.md`
4     identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5     no-regression gate, the pinnable classifier artifact, the persisted-candidate
6     substrate) already exists ? this module composes it. No new scoring math lives
7     here; everything defers to:
8
9     - `backend.eval.metrics`      ? `score`, `match_intervals`, `interval_iou`
10    - `backend.eval.training`      ? `label_candidates` (y by the same IoU matcher)
11    - `backend.eval.classifier`    ? `LogisticRegression` (the pinnable JSON model)
12    - `backend.eval.gemini_refine` ? `merge_overlaps` (the blessed window merge)
13    - `backend.eval.regression`    ? `gt_hash` (label-set fingerprint)
14    - `backend.eval.calibration`   ? `pct_improvement`
15
16    The thesis (`EXPERIMENT_HARNESS.md` §2): separate the expensive, risky DETECTION
17    (per-frame signals ? run once on the GPU/WSL box, persisted into
18    `candidate_segments.stats`) from the cheap, swappable DECISION (given those
19    signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20    given persisted candidate features it produces `List[Interval]` deterministically,
21    with no GPU and zero production risk. So most experiments are pure offline

```

```

22     re-scoring of stored data and never touch the served pipeline.
23
24     Three modes (`EXPERIMENT_HARNESS.md` §5):
25     - `compare()` ? labeled A/B vs golden labels: per-video + LOVO-honest
26       aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27       held-out loop, but variant-parameterised.
28     - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29       variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30       human spot-checks (and what two highlight reels would show side by side).
31     - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32       (exit 0/1/3); **rollback = flip the variant/config, never `git revert`.**
33
34     Pure + offline + unit-tested. The only DB-touching helper is
35     `load_video_candidates`, which reads persisted candidates via
36     `Database.query_candidate_segments`.
37     """
38     from __future__ import annotations
39
40     import glob
41     import json
42     import logging
43     import os
44     from dataclasses import dataclass, field
45     from datetime import datetime, timezone
46     from hashlib import sha1
47     from typing import Any, Callable, Dict, List, Optional, Sequence
48
49     from backend.eval.calibration import pct_improvement
50     from backend.eval.classifier import LogisticRegression
51     from backend.eval.gemini_refine import merge_overlaps
52     from backend.eval.metrics import Interval, match_intervals, score
53     from backend.eval.regression import gt_hash
54     from backend.eval.training import label_candidates
55     from backend.eval import eval_stats as _stats          # C1: CIs + paired sign test
56     from backend.eval import segmentation_metrics as _segm  # C4: F1@k + segment-count
57     ratio
58     from backend.eval.score_calibration import fit_isotonic, fit_platt # C2: calibrate cue
59     scores
60     logger = logging.getLogger(__name__)
61

```

20. user (2026-06-19T15:47:03.460Z)

```

1     """Leave-one-signal-out / leave-one-group-out ablation runner (Phase 0).
2
3     A *systematic, repeatable* version of the one-off ablation
4     (`scratch/gate_ab_ablation.py`):
5     run it and it EXCLUDES a single signal, measures whether quality dropped, and reports ?
6     per
7     signal ? the **marginal contribution** ?F1 (full set vs full-minus-that-signal) with a
8     bootstrap
9     95% CI + paired sign test + the C4 over-segmentation ratio. See
10    `scratch/ABLATION_FRAMEWORK_PLAN.md`
11    (the design) and `docs/ABLATION_LAYER.md` (the shipped note).
12
13    **Nothing here is new scoring/stats math.** It is a *pure driver* over the shipped
14    `experiment.compare` + `eval_stats` engine, exactly as `compare` was a driver over
15    `metrics`/`calibration`. For each signal `s` the marginal contribution is:
16
17    ?(s) = score(full set S) ? score(S \ {s})          # leave-one-out drop, ?F1
18    primary
19
20    operationally `experiment.compare(videos, variant_a=without_s, variant_b=full)` ? A is
21    the

```

```

16      ablated (smaller) variant, B is full, so the reported B?A delta IS ?(s), a per-video-
paired ?F1
17      with bootstrap CI + exact sign test, LOVO-honest for learned variants.
18
19      Two ablation granularities fall out of one inventory:
20
21      - **Leave-one-signal-out (LOSO)** ? remove ONE column at a time (fine-grained).
22      - **Leave-one-GROUP-out (LOGO)** ? remove a whole producer's columns at once (the
owner's
23          "is this old SIGNAL still relevant?" question at cue granularity).
24
25      A signal can be a **feature column** (person/audio/shuttle columns the learned scorer
weights) or
26      a **gate knob** (Gate-A's ``min_crossings`` threshold ? ablated by zeroing the knob, NOT
by
27      dropping a column). The runner handles both uniformly: ``without_s`` subtracts the
columns AND/OR
28      zeroes the gate knob from ``cfg.full``.
29
30      Honest small-N contract (inherited from ``eval_stats`` / ``per_user_gate``; $6 of the
plan):
31      - never a bare mean ? every row ships a CI + the exact sign test;
32      - a signal **earns its keep** only when removal drops F1 with **CI excluding 0 AND the
sign test
33          agreeing** (the ``per_user_gate`` bar, in reverse);
34      - **structural signals are exempt from auto-demotion** ? a guard whose value is in
failure modes
35          the golden corpus may lack (Gate-A vs held-shuttle FPs) is marked ``structural`` and
is NEVER
36          flagged dead weight on "no measured lift"; its row reports the (possibly ?0) golden
? honestly
37          but tags ``structural_protected``;
38      - the runner emits ``earns_keep`` / ``suggestive`` / ``inert`` /
``structural_protected`` ? it
39          MUST NOT print "irrelevant" or "proven useless" (one wash at n=6 is noise, not
proof).
40
41      Pure + offline + GPU-free. Reuses the persisted golden feature substrate
(``fusion_golden.py``
42      writes one ``<name>_golden_features.json`` per video). CLI::
43
44      python -m backend.eval.ablation --features-dir output [--pdf out.pdf]
45      """
46      from __future__ import annotations
47
48      import argparse
49      import json
50      import logging
51      from dataclasses import dataclass, field
52      from typing import Any, Dict, List, Optional, Sequence, Tuple
53
54      from backend.eval import experiment
55      from backend.eval.experiment import Variant, VideoCandidates
56
57      logger = logging.getLogger(__name__)
58
59      # --- The signal inventory (the things the runner ablates), grouped by producer.
-----
60      # Source: distill_local.FEATURE_NAMES (shuttle, includes net_crossings), fusion_features
61      # (person), audio_features (audio). The shuttle group's velocity block + duration are
the
62      # learned-column shuttle signals; net_crossings is BOTH a column and the Gate-A gate
knob.
63      SHUTTLE_COLUMNS: Tuple[str, ...] = ("duration", "peak_velocity", "mean_velocity",
64          "active_ratio", "net_crossings")

```

```

65     PERSON_COLUMNS: Tuple[str, ...] = ("players_in_court", "two_sided_motion",
"mean_player_motion",
66                                     "in_court_ratio", "shuttle_player_colocation")
67     AUDIO_COLUMNS: Tuple[str, ...] = ("audio_hit_count", "audio_hit_rate",
"audio_hit_strength_mean")
68
69     # Verdict vocabulary (the no-over-claim contract ? never "irrelevant"/"proven useless").
70     EARNS_KEEP = "earns_keep"                # CI clears 0 AND sign test agrees ? removing it
hurts
71     SUGGESTIVE = "suggestive"                # point estimate leans positive but CI spans 0
72     INERT = "inert"                          # wash this run (?0, CI spans 0) ? candidate,
not proof
73     STRUCTURAL_PROTECTED = "structural_protected"    # a guard; never auto-demoted on "no
lift"
74
75     #: Human-readable one-liners for each verdict (used in the PDF + table legend).
76     VERDICT_LABEL = {
77         EARNS_KEEP: "clears-bar",
78         SUGGESTIVE: "suggestive",
79         INERT: "inert",
80         STRUCTURAL_PROTECTED: "structural-protected",
81     }
82
83
84     @dataclass(frozen=True)
85     class Signal:
86         """One unit of ablation ? a feature column (or a named group) and/or a gate knob.
87
88         - ``name`` ? table/row identity ("person.colocation" | "gate_a" | "audio").
89         - ``group`` ? producer ("person" | "gate_a" | "audio" | "shuttle"). LOGO ablates a
whole group.
90         - ``columns`` ? the persisted feature columns this signal contributes (may be empty
for a
91         pure gate signal like Gate-A whose column is shared with the shuttle group).
92         - ``gate_knob`` ? e.g. ``"min_crossings"`` for the Gate-A threshold; ablated by
zeroing it on
93         the full variant rather than dropping a column.
94         - ``structural`` ? a guard whose value is in failure modes the golden corpus may
lack; NEVER
95         flagged dead weight on "no measured lift" (mirrors ``per_user_gate``
``structural=True``).
96         """
97
98         name: str
99         group: str
100         columns: Tuple[str, ...] = ()
101         gate_knob: Optional[str] = None
102         structural: bool = False
103
104
105     @dataclass
106     class AblationConfig:
107         """How to run an ablation over a corpus.
108
109         - ``full`` ? the full-set reference variant (B). For the litmus Gate-A reproduction
this is
110         the STATIC ``Variant(min_crossings=2)`` (no classifier); for a learned ablation it
carries
111         ``feature_names`` = all columns and the gates ON.
112         - ``signals`` ? the inventory to ablate.
113         - ``granularity`` ? ``"signal"`` (LOSO, one column-set at a time) or ``"group"``
(LOGO).
114         - ``iou`` ? temporal-IoU match threshold.
115         - ``honest`` ? LOVO for learned variants (the number to trust).
116         - ``seed`` ? threaded into the ``eval_stats`` bootstrap (determinism).

```

```

117     - ``p_threshold`` ? sign-test significance bar for the ``earn_keep`` verdict.
118     ""
119
120     full: Variant
121     signals: List[Signal]
122     granularity: str = "group"
123     iou: float = 0.5
124     honest: bool = True
125     seed: int = 0
126     p_threshold: float = 0.05
127
128
129     @dataclass
130     class AblationRow:
131         """One signal's marginal-contribution row (?s) + CI + sign test + C4 + a
132         verdict)."""
133         signal: str
134         group: str
135         structural: bool
136         dropped_columns: List[str]
137         gate_knob: Optional[str]
138         n: int
139         delta_f1: float
140         ci_low: float
141         ci_high: float
142         ci_excludes_zero: bool
143         sign_wins: int
144         sign_losses: int
145         sign_p_value: float
146         f1_full: float
147         f1_ablated: float
148         seg_ratio_full: float
149         seg_ratio_ablated: float
150         verdict: str
151         structural_protected: bool
152
153         def as_dict(self) -> Dict[str, Any]:
154             return {
155                 "signal": self.signal,
156                 "group": self.group,
157                 "structural": self.structural,
158                 "dropped_columns": list(self.dropped_columns),
159                 "gate_knob": self.gate_knob,
160                 "n": self.n,
161                 "delta_f1": self.delta_f1,
162                 "ci_low": self.ci_low,
163                 "ci_high": self.ci_high,
164                 "ci_excludes_zero": self.ci_excludes_zero,
165                 "sign_test": {"wins": self.sign_wins, "losses": self.sign_losses,
166                             "p_value": self.sign_p_value},
167                 "f1_full": self.f1_full,
168                 "f1_ablated": self.f1_ablated,
169                 "segment_count_ratio_full": self.seg_ratio_full,
170                 "segment_count_ratio_ablated": self.seg_ratio_ablated,
171                 "verdict": self.verdict,
172                 "structural_protected": self.structural_protected,
173             }
174
175
176     @dataclass
177     class AblationReport:
178         """All signals' rows + the run's provenance ? a structured, JSON-able ablation
179         result."""

```

```

180     granularity: str
181     iou: float
182     num_videos: int
183     honest_lovo: bool
184     full_set_signals: List[str]
185     full_spec_hash: str
186     label_set_hash: str
187     rows: List[AblationRow] = field(default_factory=list)
188
189     def sorted_rows(self) -> List[AblationRow]:
190         """Rows by marginal ?F1 desc ? the contribution ranking (the plan's table
order)."""
191         return sorted(self.rows, key=lambda r: r.delta_f1, reverse=True)
192
193     def as_dict(self) -> Dict[str, Any]:
194         return {
195             "granularity": self.granularity,
196             "iou": self.iou,
197             "num_videos": self.num_videos,
198             "honest_lovo": self.honest_lovo,
199             "full_set_signals": list(self.full_set_signals),
200             "full_spec_hash": self.full_spec_hash,
201             "label_set_hash": self.label_set_hash,
202             "rows": [r.as_dict() for r in self.sorted_rows()],
203         }
204
205
206     # ----- the driver
207
208
209     def _ablated_variant(full: Variant, columns: Sequence[str], gate_knob: Optional[str]) ->
Variant:
210         """Build the leave-one-out (smaller) variant from ``full``: drop ``columns`` from
its
211         ``feature_names`` and/or zero the named gate knob. Pure ? derives a new Variant,
never
212         mutates ``full``."""
213         cols = set(columns)
214         fn = full.feature_names
215         ablated_fn = [c for c in fn if c not in cols] if fn is not None else fn
216         overrides: Dict[str, Any] = {"feature_names": ablated_fn}
217         if gate_knob == "min_crossings":
218             overrides["min_crossings"] = 0
219         elif gate_knob == "min_players":
220             overrides["min_players"] = 0
221         elif gate_knob is not None:
222             raise ValueError(f"unknown gate_knob {gate_knob!r} (expected
'min_crossings'/'min_players')")
223         dropped_label = ",".join(sorted(cols)) or (gate_knob or "?")
224         return experiment.Variant.from_dict(**full.to_dict(), **overrides,
225             "name": f"?{dropped_label}")
226
227
228     def _classify(structural: bool, ci_excludes_zero: bool, sign_wins: int, sign_losses:
int,
229     sign_p: float, p_threshold: float) -> Tuple[str, bool]:
230         """The honest verdict (no over-claim). Returns ``(verdict, structural_protected)``.
231
232         A structural guard is reported but never demoted on "no lift". Otherwise: removing
the
233         signal *earns its keep* when the ?F1 CI clears 0 AND the sign test agrees (majority
worse on
234         removal, ``p <= p_threshold``); a positive-leaning but CI-spanning row is
``suggestive``; a
235         wash this run is ``inert`` (a candidate, never "proven useless" at n=6)."""

```

```

236     if structural:
237         return STRUCTURAL_PROTECTED, True
238     sign_agrees = (sign_p <= p_threshold) and (sign_wins > sign_losses)
239     if ci_excludes_zero and sign_agrees:
240         return EARNES_KEEP, False
241     if sign_wins > sign_losses:          # leans toward "removal hurts" but evidence is
thin
242         return SUGGESTIVE, False
243     return INERT, False
244
245
246 def ablate_one(videos: Sequence[VideoCandidates], cfg: AblationConfig,
247               signal: Signal) -> AblationRow:
248     """One ablation row: marginal ?(s) + CI + sign test + C4 + a verdict.
249
250     Drops ``signal``'s columns / zeroes its gate knob from ``cfg.full`` to form the
ablated
251     variant A, then calls ``experiment.compare(A, full)`` so the reported B?A ?F1 IS
252     ?(s).
253     Reuses ``compare`` verbatim ? learned variants stay LOVO-honest, the C1+C4 honest
stats
254     come for free."""
255     ablated = _ablated_variant(cfg.full, signal.columns, signal.gate_knob)
256     res = experiment.compare(videos, ablated, cfg.full, iou=cfg.iou, honest=cfg.honest,
257                             honest_stats=True)
258     # Re-pair the per-video ?F1 with the configured bootstrap seed for determinism (the
default
259     # honest block uses seed=0; thread cfg.seed explicitly so a run is reproducible).
260     from backend.eval.eval_stats import paired_delta_ci
261     by_a = {p["video"]: p["f1"] for p in res["variant_a"]["per_video"]}
262     by_b = {p["video"]: p["f1"] for p in res["variant_b"]["per_video"]}
263     vids = [p["video"] for p in res["variant_a"]["per_video"]]
264     pd = paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids],
seed=cfg.seed)
265     st = pd["sign_test"]
266     sign_wins, sign_losses = int(st["wins"]), int(st["losses"])
267     sign_p = float(st["p_value"])
268     verdict, protected = _classify(signal.structural, bool(pd["ci_excludes_zero"]),
269                                   sign_wins, sign_losses, sign_p, cfg.p_threshold)
270
271     seg_full = res["honest"]["segmentation"]["variant_b"]["segment_count_ratio"]
272     seg_abl = res["honest"]["segmentation"]["variant_a"]["segment_count_ratio"]
273
274     return AblationRow(
275         signal=signal.name,
276         group=signal.group,
277         structural=signal.structural,
278         dropped_columns=list(signal.columns),
279         gate_knob=signal.gate_knob,
280         n=int(pd["n"]),
281         delta_f1=float(pd["mean_delta"]),
282         ci_low=float(pd["ci_low"]),
283         ci_high=float(pd["ci_high"]),
284         ci_excludes_zero=bool(pd["ci_excludes_zero"]),
285         sign_wins=sign_wins,
286         sign_losses=sign_losses,
287         sign_p_value=sign_p,
288         f1_full=float(res["variant_b"]["aggregate"]["f1"]),
289         f1_ablated=float(res["variant_a"]["aggregate"]["f1"]),
290         seg_ratio_full=float(seg_full),
291         seg_ratio_ablated=float(seg_abl),
292         verdict=verdict,
293         structural_protected=protected,
294     )

```

```

295
296     def _collapse_to_groups(signals: Sequence[Signal]) -> List[Signal]:
297         """Collapse a LOSO inventory into one Signal per producer GROUP (LOGO): union the
columns,
298         OR the gate knob, OR the structural flag. First-seen group order is preserved."""
299         order: List[str] = []
300         by_group: Dict[str, List[Signal]] = {}
301         for s in signals:
302             if s.group not in by_group:
303                 order.append(s.group)
304                 by_group[s.group] = []
305             by_group[s.group].append(s)
306         out: List[Signal] = []
307         for g in order:
308             members = by_group[g]
309             cols: Tuple[str, ...] = tuple(dict.fromkeys(c for m in members for c in
m.columns))
310             knob = next((m.gate_knob for m in members if m.gate_knob), None)
311             structural = any(m.structural for m in members)
312             out.append(Signal(name=g, group=g, columns=cols, gate_knob=knob,
structural=structural))
313         return out
314
315
316     def run_ablation(videos: Sequence[VideoCandidates], cfg: AblationConfig) ->
AblationReport:
317         """Loop the inventory at the configured granularity ? an ``AblationReport`` (sorted
table).
318
319         A pure driver: each row is one ``experiment.compare(without_s, full)``.
``granularity="group"``
320         (LOGO) ablates a whole producer's columns at once; ``"signal"`` (LOSO) one column-
set at a
321         time. The full reference is scored once implicitly per row (cheap re-scoring;
detection is
322         already persisted)."""
323         if len(videos) < 1:
324             raise experiment.ExperimentError("run_ablation needs at least one labeled
video")
325         if cfg.granularity not in ("signal", "group"):
326             raise ValueError(f"granularity must be 'signal' or 'group', got
{cfg.granularity!r}")
327
328         inventory = cfg.signals if cfg.granularity == "signal" else
_collapse_to_groups(cfg.signals)
329         rows = [ablate_one(videos, cfg, s) for s in inventory]
330
331         # Provenance: a full-set evaluation gives the honest-LOVO flag + the label-set
fingerprint.
332         full_eval = experiment.evaluate_variant(videos, cfg.full, iou=cfg.iou,
honest=cfg.honest)
333         from backend.eval.regression import gt_hash
334         all_gts = [iv for vc in videos for iv in (vc.gts or [])]
335         return AblationReport(
336             granularity=cfg.granularity,
337             iou=cfg.iou,
338             num_videos=len(videos),
339             honest_lovo=bool(full_eval["honest_lovo"]),
340             full_set_signals=[s.group for s in _collapse_to_groups(cfg.signals)],
341             full_spec_hash=cfg.full.spec_hash(),
342             label_set_hash=gt_hash(all_gts),
343             rows=rows,
344         )
345
346

```

```

347 # ----- default
inventories
348
349
350 def default_signals() -> List[Signal]:
351     """The current 4-group inventory (LOSO granularity): shuttle (velocity block +
duration),
352     gate_a (the structural net-crossings gate AND its column), person, audio.
353
354     Gate-A is marked ``structural`` (the held-shuttle FP guard) so it is reported but
never
355     flagged dead weight on "no measured lift". It ablates via the ``min_crossings`` knob
AND by
356     dropping the ``net_crossings`` column, so the leave-one-out genuinely removes the
whole cue."""
357     return [
358         Signal("shuttle.duration", "shuttle", ("duration",)),
359         Signal("shuttle.velocity", "shuttle", ("peak_velocity", "mean_velocity",
"active_ratio")),
360         Signal("gate_a", "gate_a", ("net_crossings",), gate_knob="min_crossings",
structural=True),
361         Signal("person", "person", PERSON_COLUMNS),
362         Signal("audio", "audio", AUDIO_COLUMNS),
363     ]
364
365
366 def static_gate_a_signal() -> Signal:
367     """The Gate-A-only structural signal (a pure gate knob over the recall-oriented
proposal
368     set). Ablating it against the static ``Variant(min_crossings=2)`` full reproduces
the
369     hand-measured Gate-A result (?F1 ? +0.074, 6/6, p ? 0.031) ? the framework's litmus
test."""
370     return Signal("gate_a", "gate_a", (), gate_knob="min_crossings", structural=True)
371
372
373 def litmus_config() -> AblationConfig:
374     """The Gate-A reproduction config: full = STATIC ``Variant(min_crossings=2)`` (no
learned
375     classifier), inventory = the single Gate-A gate signal. Ablating it = CONTROL+gate
vs CONTROL
376     ? exactly the historical measurement (``scratch/gate_ab_ablation.py``)."""
377     full = experiment.Variant(name="control_gate_a", min_crossings=2)
378     return AblationConfig(full=full, signals=[static_gate_a_signal()],
granularity="signal")
379
380
381 # ----- corpus
I/O
382
383
384 def load_videos(features_dir: str) -> List[VideoCandidates]:
385     """Load every FULL-FUSION-SCHEMA ``*_golden_features.json`` into a
``VideoCandidates``
386     (intervals + shuttle + person + audio columns + golden gts). Audio columns are
optional
387     (older JSONs predate the audio augment) ? a missing block defaults those columns to
0.0 so
388     the loader never crashes, mirroring ``experiment._feature_column``'s tolerance.
389
390     Leaner velocity-windowing-only files (no ``shuttle``/``person`` cue dicts ? the
schema the
391     R3/R4 corpus-growth path persists) are SKIPPED, not crashed on, so a mixed corpus
loads the
392     ablatable subset cleanly (see ``_is_fusion_schema``)."""

```

```

393     return experiment.load_golden_videos(
394         features_dir,
395         shuttle_columns=SHUTTLE_COLUMNS,
396         person_columns=PERSON_COLUMNS,
397         audio_columns=AUDIO_COLUMNS,
398         audio_policy="tolerate-missing",
399         skip_logger=logger,
400         skip_message="ablation.load_videos: skipping %s ? windowing-only schema "
401             "(no shuttle/person cue dicts), not an ablatable fusion-feature
file",
402     )
403
404
405     def learned_full_variant(*, threshold: float = 0.5, min_crossings: int = 0) -> Variant:
406         """The learned full-set reference: all 13 columns (shuttle 5 + person 5 + audio 3)
weighted
407         by the LOVO logistic scorer. Used for the LOSO/LOGO contribution table over the
learned path
408         (distinct from the static Gate-A litmus)."""
409         cols = list(SHUTTLE_COLUMNS) + list(PERSON_COLUMNS) + list(AUDIO_COLUMNS)
410         return experiment.Variant(name="full_learned", feature_names=cols,
411             threshold=threshold, min_crossings=min_crossings)
412
413
414     # ----- table
rendering
415
416
417     def format_table(report: AblationReport) -> str:
418         """Sorted ASCII/markdown ablation table (by marginal ?F1 desc ? the contribution
ranking)."""
419         rows = report.sorted_rows()
420         header = (f"Ablation report ? full set = {{{', '.join(report.full_set_signals)}}}"
421             f" . n={report.num_videos} . IoU {report.iou}"
422             f" . {'LOVO-honest' if report.honest_lovo else 'static'}"
423             f" . {report.granularity.upper()}")
424         cols = ["signal", "?F1(full?ablated)", "95% CI", "sign test", "CI?0", "verdict"]
425         widths = [18, 17, 18, 14, 5, 22]
426
427         def _fmt_row(cells: Sequence[str]) -> str:
428             return " ".join(c.ljust(w) for c, w in zip(cells, widths))
429
430         lines = [header, "", _fmt_row(cols), _fmt_row(["-" * w for w in widths])]
431         for r in rows:
432             ci = f"[{r.ci_low:+.3f}, {r.ci_high:+.3f}]"
433             sign = f"{r.sign_wins}W/{r.sign_losses}L p={r.sign_p_value:.3f}"
434             verdict = VERDICT_LABEL.get(r.verdict, r.verdict)
435             lines.append(_fmt_row([r.signal, f"{r.delta_f1:+.3f}", ci, sign,
436                 "yes" if r.ci_excludes_zero else "no", verdict]))
437         return "\n".join(lines)
438
439
440     # ----- PDF
export
441
442     # The ~116-line reportlab PDF builder lives in a sibling module to keep this file a pure
driver;
443     # re-exported here so ``ablation.export_pdf`` stays a valid public name. Placed AFTER
the
444     # dataclasses/constants it consumes (``ablation_pdf`` imports FROM this module) to avoid
a cycle.
445     from backend.eval.ablation_pdf import export_pdf as export_pdf # noqa: E402
446
447
448     # -----

```

```

CLI
449
450
451     def main() -> None:
452         ap = argparse.ArgumentParser(
453             description="LOSO/LOGO ablation runner over the golden feature substrate (pure
driver "
454                 "over experiment.compare + eval_stats; no new scoring math).")
455         ap.add_argument("--features-dir", default="output",
456             help="dir of *_golden_features.json (the persisted golden
substrate)")
457         ap.add_argument("--granularity", choices=("signal", "group"), default="group",
458             help="leave-one-SIGNAL-out (column) or leave-one-GROUP-out
(producer)")
459         ap.add_argument("--iou", type=float, default=0.5)
460         ap.add_argument("--seed", type=int, default=0, help="bootstrap seed (determinism)")
461         ap.add_argument("--litmus", action="store_true",
462             help="reproduce the static Gate-A measurement (CONTROL vs
min_crossings=2) "
463                 "instead of the learned full-set ablation")
464         ap.add_argument("--pdf", default=None, help="also write a PDF ablation report to
this path")
465         ap.add_argument("--json", default=None, help="also write the structured report JSON
here")
466         args = ap.parse_args()
467
468         videos = load_videos(args.features_dir)
469         if len(videos) < 2:
470             raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} "
471                             f"(found {len(videos)})")
472
473         if args.litmus:
474             cfg = litmus_config()
475             cfg.iou, cfg.seed = args.iou, args.seed
476         else:
477             cfg = AblationConfig(full=learned_full_variant(), signals=default_signals(),
478                                 granularity=args.granularity, iou=args.iou, seed=args.seed)
479
480         report = run_ablation(videos, cfg)
481         print(format_table(report))
482
483         if args.json:
484             with open(args.json, "w", encoding="utf-8") as f:
485                 json.dump(report.as_dict(), f, indent=2)
486                 print(f"\nwrote report JSON -> {args.json}")
487         if args.pdf:
488             export_pdf(report, args.pdf)
489             print(f"wrote PDF report -> {args.pdf}")
490
491
492     if __name__ == "__main__":
493         main()
494

```

21. assistant (2026-06-19T15:47:07.987Z)

Now let me verify the `experiment.predict` function at line 213 (ANALYZERS doc claims it treats `net\_crossings` and `players\_in\_court` as feature columns with `\_gate\_mask`), and check the corpus size question. The docs reference "n=6 golden videos" in several places but the gap-closure doc says n=13. Let me check the actual predict function and the corpus state.

22. user (2026-06-19T15:47:09.915Z)

```

130         "name": self.name,
131         "segmenter": self.segmenter,

```

```

132         "config_overrides": self.config_overrides,
133         "cue_flags": self.cue_flags,
134         "min_crossings": self.min_crossings,
135         "min_players": self.min_players,
136         "classifier_model": self.classifier_model,
137         "feature_names": self.feature_names,
138         "threshold": self.threshold,
139         "merge_gap": self.merge_gap,
140         "tune_threshold": self.tune_threshold,
141         "calibrate": self.calibrate,
142     }
143
144     @classmethod
145     def from_dict(cls, d: dict) -> "Variant":
146         known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
147         return cls(**{k: v for k, v in d.items() if k in known})
148
149     def spec_hash(self) -> str:
150         """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
151         and makes a result reproducible. The pinned **control** variant always hashes
152         the
153         same, so a regression is always traceable to a recipe change, never to drift."""
154         blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
155         return sha1(blob.encode()).hexdigest()[:12]
156
157     #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
158     #: filter. Always the comparison reference; "rollback" = make this the served variant.
159     CONTROL = Variant(name="control")
160
161
162     # ----- persisted candidates
163
164
165     @dataclass
166     class VideoCandidates:
167         """One video's persisted detection, ready for offline re-scoring.
168
169         ``intervals`` are the candidate windows; ``features`` is column-major
170         (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
171         ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
172         with ``load_video_candidates``, or directly in tests.
173         """
174
175         name: str
176         intervals: List[Interval]
177         features: Dict[str, List[float]] = field(default_factory=dict)
178         gts: Optional[List[Interval]] = None
179
180
181     def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
182         """True iff every candidate row carries the per-cue feature dicts the golden-feature
183         loaders
184         read (``shuttle`` + ``person``; ``audio`` stays optional ? it is added later by
185         ``fusion_audio.augment`` and older JSONs predate it).
186
187         The velocity-windowing / distill corpus-growth path persists a LEANER
188         ``*_golden_features.json`` row (``start``/``end`` + velocity stats, no cue dicts)
189         for clips
190         that never went through fusion feature extraction. The loaders that build cue-rich
191         ``VideoCandidates`` from these files ? ``ablation.load_videos``,
192         ``fusion_compare.load_videos``
193         and ``fusion_audio.load_videos_with_audio`` ? SKIP those rather than crash on
194         ``c["shuttle"]``,
195         so a MIXED corpus (some fusion files, some windowing-only) loads the fusion subset

```

```

cleanly.
192     An empty candidate list is not a fusion file either.
193
194     Single source of truth for the three loaders (consolidated 2026-06-18 after PRs
#209/#210
195     each landed an independent copy)."""
196     return bool(candidates) and all("shuttle" in c and "person" in c for c in
candidates)
197
198
199     def load_golden_videos(
200         features_dir: str,
201         *,
202         shuttle_columns: Sequence[str],
203         person_columns: Sequence[str],
204         audio_columns: Sequence[str] = (),
205         audio_policy: str = "omit",
206         skip_logger: logging.Logger = logger,
207         skip_message: str,
208         no_audio_error: str = "{path} has no audio features",
209     ) -> List["VideoCandidates"]:
210         """Shared golden-feature corpus loader behind the three structurally-identical
loaders
211         (`ablation.load_videos` / `fusion_compare.load_videos` /
`fusion_audio.load_videos_with_audio`).
212
213         Reads every FULL-FUSION-SCHEMA ``*_golden_features.json`` in ``features_dir`` into a
214         ``VideoCandidates`` (intervals + the caller-supplied feature columns + golden gts).
Leaner
215         velocity-windowing-only files (no ``shuttle``/``person`` cue dicts ? the schema the
216         corpus-growth path persists) are SKIPPED, not crashed on (see
``_is_fusion_schema``), so a
217         MIXED corpus loads the fusion subset cleanly.
218
219         The column lists are ALWAYS supplied by the caller ? this loader never hardcodes
columns. The

```

23. user (2026-06-19T15:47:10.849Z)

```

92:     ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
96:     whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
282: def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
287:     col = _feature_column(vc, "net_crossings")
290:     col = _feature_column(vc, "players_in_court")
308:     mask = _gate_mask(variant, vc)

```

24. assistant (2026-06-19T15:47:17.798Z)

The ``predict`/`_gate_mask`` confirms ``experiment.py`` treats ``net_crossings`` and ``players_in_court`` as feature columns (line 282-290). The ANALYZERS doc references to line 213 are now off (predict is no longer at 213, load_golden_videos is) but that's a line-number drift, not a build-status staleness claim ? out of my slice's scope (which is signal build-status).

Now let me focus on the core staleness in my slice. The SEED finding is the key one in MULTI_SIGNAL_FUSION_PLAN. Let me re-examine the specific stale claims. The doc §4 table marks ``shuttle_player_colocation`` as `***NEW**` (needs the person cue) and §3.2 still lists it as a feature "to add". Let me check the exact text and also verify the HOW_RALLY doc's "named-but-unbuilt Gate B" claim.

Let me re-read the precise stale fragments in MULTI_SIGNAL_FUSION_PLAN §3.2, §4, and the HOW doc.

25. user (2026-06-19T15:47:19.192Z)

```

42         person cue and generalizes the fusion seam so future cues are "register a producer,"
not "re-touch
43         the segmenter."
44         - **backend/pipeline/detectors/rally_gate.py` already exists** ? the net-crossing gate,
explicitly
45         labelled in its own docstring as **Gate A in the over-fire fix spectrum (**B = player-
stance state
46         machine, future**)." ** The person cue in this plan **is that named-but-unbuilt Gate
B.**
47

```

26. assistant (2026-06-19T15:47:26.800Z)

Now I have a clear view. Let me analyze the staleness claims carefully against the code:

****Key code facts established:****

1. ``shuttle_player_colocation`` IS BUILT (``fusion_features.py:168``, in ``PERSON_FEATURE_NAMES``, line 197), computed in ``compute_person_features`` (line 213), wired in ``fusion_hybrid.py`` under ``cue_flags.person`` ? but default-OFF, mock-tested, unmeasured on golden.
2. Gate B (person/occupancy) IS BUILT ? ``fusion_hybrid.py::_select_for_handoff`` implements ``min_players`` gating (lines 159, 166); ``FusionConfig.min_players`` exists (`models.py:108`). Person features incl. occupancy are built in ``fusion_features.py``.
3. The full person cue (Gate B) is built and wired, default-OFF, unmeasured.

Now let me identify the stale statements. Let me check whether HOW_RALLY's "named-but-unbuilt Gate B" is stale, and the MULTI_SIGNAL doc's "NEW (needs the person cue)" / "Gate B ? the person cue, NEW" / "named-but-unbuilt Gate B".

Let me look at the §3.1 and the §1 lines more precisely for the "NEW" tags.

27. user (2026-06-19T15:47:27.400Z)

```

113
114     ``
115     keep window ? net_crossings ? 2                (Gate A ? EXISTS in rally_gate.py, eval-
only today)
116             ? (?? in-court players ? recent track)  (Gate B ? the person cue, NEW)
117     ``
118
119     Gate A kills the service-feed / held-shuttle case structurally; Gate B adds court-
occupancy evidence
120     for the cases a single trajectory can still fool (e.g. a knock-up that does cross the
net). No training

```

28. assistant (2026-06-19T15:47:36.362Z)

Now I have all the evidence I need. Let me also verify the gap-closure doc's Lever A claim that ``shuttle_player_colocation`` is ``[design]``. Looking back at line 56 of RALLY_DETECTION_GAP_CLOSURE.md: "``shuttle_player_colocation`` (shuttle box ? person box = held) ``[design]`` from the fusion plan §4" and line 77 "Extract person-velocity + ``shuttle_player_colocation`` on the golden clips (default-OFF features)".

The gap-closure doc tags ``shuttle_player_colocation`` as ``[design]`` and "new feature" in the reuse map. But the feature IS built in code (``fusion_features.py:168``). However, I need to be careful: the gap-closure doc's framing is about *extracting it on the golden clips* (P3 deliverable), which IS genuinely not-yet-done (unmeasured on golden). The ``[design]`` in Lever A refers to the player-state rally-END *rule* being design. Let me re-read line 56 and 61 carefully and line 58.

Line 58: "the FusionSegmenter feature path" ? accurate. Line 61 reuse map: "``shuttle_player_colocation`` (new feature)" ? this calls it "new" but it IS built. Let me check this is genuinely a build-status contradiction vs. "new to the golden-scoring path".

Let me look at the exact gap-closure text once more around lines 54-61.

29. user (2026-06-19T15:47:37.202Z)

54 - **Lever A ? player-state rally-END rule** (targets **G3 + G2**) ``[design]``. Generalize R4 from shuttle-only to **shuttle OR bilateral player motion**: end the rally at the last frame with competitive evidence from **either**, and **hard-terminate** on a "shuttle picked up" event. Two cheap detectors, no new model required:

55 - **Player kinematics ? person velocity drops / players go stationary after a point** (the casual returner taps it while the other walks). **Reuse**: the person cue producer (torchvision + ByteTrack) in ``yolo_hybrid`` (fusion P1, ``[verified]`` exists).

56 - **Shuttle-in-hand / out-of-play ? `shuttle\_player\_colocation`** (shuttle box ? person box = held) ``[design]`` from the fusion plan §4, **or** the no-new-model proxy: **sustained WASB Visibility=0** after the last fast stroke (held shuttle goes dark). Use it as a **termination anchor + reverse-timeline search** to the true end (last net-crossing / last fast-shuttle / last bilateral motion) ? the owner's idea, refined: the hold event is **late**, so anchor-then-backtrack, don't use its timestamp as the end.

57 - **Lever B ? recall** (targets **G1**) ``[design]``. Both systems miss real rallies ? detector sensitivity: proposal-windowing / velocity-threshold tuning, and the **complementary-fusion** angle below. Measure recall floor per video; the highest-leverage labels are the **missed** rallies (active learning, RALLY_QUALITY_RESEARCH §4.3).

58 - **Lever C ? signal/ensemble fusion** (targets **G1**) ``[design]``. Local & Gemini (and shuttle vs person cues) hit **different** rallies ? a union/ensemble lifts recall. Already-built scaffolding: ``rally_gate.py`` **Gate A** (net-crossings, structural rally-state, ``[verified]`` built, eval-only ? wire to prod = cheapest precision win for G2) and the FusionSegmenter feature path.

59 - **Pure-precision cleanup** (**G2**): ``rally_gate`` Gate A kills single-feed / held-shuttle **starts** structurally (?1 net crossings vs a real rally's several) ? the cheapest G2 win and independent of the person stack.

60

61 **Reuse map** (what's new vs there): ``rally_gate.py::gate_windows`` (Gate A, built) · ``tracknet_runner`` R4 end-trim (extend, don't replace) · ``backend/eval/windowing.py`` (preset knobs) · person producer in ``yolo_hybrid`` (built) · ``shuttle_player_colocation`` (new feature) · ``scratch/ingest_golden.py`` + ``scratch/boxhill17_score_golden.py`` (the per-video golden scoring loop, ``[verified]``).

30. assistant (2026-06-19T15:47:46.246Z)

Now I have all the evidence to make precise, defensible findings. Let me analyze the staleness in the gap-closure doc carefully:

- Line 56: ``shuttle_player_colocation ... [design]`` ? the **colocation** feature itself is BUILT in code (``fusion_features.py:168``). The ``[design]`` tag is on the feature, which is built. This is stale per the audit rule ("``[design]`` is stale ONLY if the thing is actually BUILT in code"). However, the broader Lever A (player-state rally-END **rule**) is genuinely design ? but the ``[design]`` tag is attached specifically to ``shuttle_player_colocation``. The feature is built; what's not done is wiring it into the rally-END rule and measuring it on golden.

- Line 61: "``shuttle_player_colocation`` (new feature)" in the reuse-map "what's new vs there" ? this lists it as new (not-built), contradicting ``fusion_features.py:168``.

Both are surgical fixes that preserve the doc's intent (the **rule** using it is design; the **measurement on golden** is undone) while correcting the build-status claim.

Now let me also re-confirm the HOW_RALLY_DETECTION_WORKS "named-but-unbuilt Gate B" claim ? Gate B (person cue + occupancy gating) IS built in ``fusion_hybrid.py``. Let me verify there are no other docs referencing the corpus size as the spec. Let me do a final grep for "unbuilt" and "not built" and "NEW" across my target files.

31. user (2026-06-19T15:47:48.830Z)

docs\HOW_RALLY_DETECTION_WORKS.md:60: ADR-007's parked #2 and ``rally_gate.py``'s named-but-unbuilt "Gate B").

docs\MULTI_SIGNAL_FUSION_PLAN.md:46: machine, future**)." The person cue in this plan **is** that named-but-unbuilt Gate B."

docs\MULTI_SIGNAL_FUSION_PLAN.md:116: ? (?2 in-court players ? recent track)
(Gate B ? the person cue, NEW)

docs\MULTI_SIGNAL_FUSION_PLAN.md:133:**Person features to add (~4, deliberately few):**

32. assistant (2026-06-19T15:48:05.813Z)

Now I have a complete picture of the staleness items in my slice. Let me analyze each candidate carefully against the audit rules.

****Confirmed code facts:****

- `shuttle\_player\_colocation` BUILT: `fusion\_features.py:168`, in `PERSON\_FEATURE\_NAMES` (line 197), computed in `compute\_person\_features` (line 213).
- Person cue / Gate B BUILT and wired: `fusion\_hybrid.py::\_select\_for\_handoff` (`min\_players` gating, lines 159/166), `\_person\_features` (lines 111-139),
`FusionConfig.min\_players`/`cue\_flags`/`court\_polygon` (models.py:84-114). All default-OFF, mock-tested, unmeasured on golden.

****Now the staleness candidates:****

1. ****HOW_RALLY line 60****: "`rally\_gate.py`'s named-but-unbuilt 'Gate B'" ? Gate B (the person cue) IS BUILT in `fusion\_hybrid.py`. STALE.
2. ****MULTI_SIGNAL line 46****: "The person cue in this plan ****is that named-but-unbuilt Gate B.****" ? Gate B is now built. STALE. (But note: §10 P2 already marks the FusionSegmenter DONE, so §1's "named-but-unbuilt" contradicts the same doc's own P2-DONE status.)
3. ****MULTI_SIGNAL line 116****: "(Gate B ? the person cue, NEW)" ? Gate B built. The "NEW" tag is stale.
4. ****MULTI_SIGNAL line 178**** (§4 table): `shuttle\_player\_colocation` ... ****NEW**** (needs the person cue)" ? the feature is built. STALE.
5. ****MULTI_SIGNAL line 133****: ****Person features to add (~4, deliberately few):**** ? these are now built. Borderline ? this is a plan section describing the design; the §10 P2-DONE note already records they're shipped. I'll flag it as low severity since "to add" frames them as not-yet-built.
6. ****GAP_CLOSURE line 56****: `shuttle\_player\_colocation` ... [design] ? feature is built. STALE per the [design] rule.
7. ****GAP_CLOSURE line 61****: "`shuttle\_player\_colocation` (new feature)" ? built. STALE.

Let me be careful with #2, #3, #4 ? the MULTI_SIGNAL doc's §1/§3/§4 are the "Context/design" narrative, but they make build-status claims ("named-but-unbuilt", "NEW") that directly contradict the code AND the doc's own §10 P2-DONE. These are legitimate. Let me make the surgical edits preserving voice.

Let me draft the corrected_text for each. For #2 (line 46), the minimal fix changes "named-but-unbuilt Gate B" to reflect it's now built (P2). Let me verify line 178 exact text.

33. user (2026-06-19T15:48:06.558Z)

```
176      |---|---|---|---|
177      | `net_crossings` | ? 1 (usually several) | ~0 (held) / ~1 (single feed) | **EXISTS**
(`rally_gate.py`); eval-only ? wire to prod |
178      | `shuttle_player_colocation` | low (free space) | high (near a hand) | **NEW** (needs
the person cue) |
179      | `direction_reversals` | several (racket contacts) | ~0 | derivable from trajectory;
partial overlap with crossings |
180      | `peak_shuttle_speed` | high | walking speed | derivable from existing velocity |
```

34. assistant (2026-06-19T15:48:15.225Z)

I now have all evidence confirmed. Note that the `net\_crossings` row at line 177 still says "eval-only ? wire to prod" ? but `FusionConfig.min\_crossings` (models.py:105) now wires Gate A to the served handoff path (`\_select\_for\_handoff`, fusion_hybrid.py:164). That's also stale. The doc's own §1 (line 60-64) already corrected this ("P2 closed this: `FusionConfig.min\_crossings` now exists"), so line 177's "eval-only ? wire to prod" contradicts both the code and the doc's own §1. Let me include that.

Let me also double-check: does `net\_crossings` line 115 in §3.1 ("Gate A ? EXISTS in rally_gate.py, eval-only today") contradict the served wiring? Yes ? but that's inside a design pseudocode block showing the gate composition. It says "eval-only today" which is now false (served gate exists). However, line 60-64 of the same doc explicitly walks through this and the §3.1 block is illustrating the *concept*. The line 177 table entry "EXISTS ...; eval-only ? wire to prod" is a status column making a direct build-status claim, so it's the cleaner finding. I'll flag line 115 as well but lower severity since it's pseudocode annotation.

Let me compile the structured findings now.

35. user (2026-06-19T15:48:45.564Z)

Structured output provided successfully

36. assistant (2026-06-19T15:49:04.138Z)

I have completed the staleness audit of my slice (rally signal/feature build-status). Here is a summary of the verified findings.

Verified code facts (the ground truth)

- `shuttle\_player\_colocation` IS BUILT: `backend/pipeline/detectors/fusion\_features.py:168`, listed in `PERSON\_FEATURE\_NAMES` (line 197), computed by `compute\_person\_features` (line 213).
- The full person cue / Gate B IS BUILT and wired: `fusion\_hybrid.py::\_person\_features` (lines 111-139) + `\_select\_for\_handoff` `min\_players` gate (line 166); config in `models.py` `FusionConfig` (`cue\_flags`, `min\_players`, `court\_polygon`, lines 84-114). All default-OFF, mock-tested, unmeasured on golden.
- Served Gate A IS WIRED: `FusionConfig.min\_crossings` (`models.py:105`) applied in `\_select\_for\_handoff` (line 164) ? not eval-only anymore.

8 staleness findings (all code-contradicting)

1. **HOW_RALLY_DETECTION_WORKS.md:60** ? calls Gate B "named-but-unbuilt"; it's built.
2. **MULTI_SIGNAL_FUSION_PLAN.md:46** ? "named-but-unbuilt Gate B"; contradicts code AND the doc's own §10 P2-DONE.
3. **MULTI_SIGNAL_FUSION_PLAN.md:116** ? "(Gate B ? the person cue, NEW)"; built.
4. **MULTI_SIGNAL_FUSION_PLAN.md:178** (§4 table) ? `shuttle\_player\_colocation` tagged **NEW** (needs the person cue); built (the SEED finding ? high severity).
5. **MULTI_SIGNAL_FUSION_PLAN.md:133/141** ? "Person features to add"; all five are built (low severity).
6. **MULTI_SIGNAL_FUSION_PLAN.md:177** (§4 table) ? `net\_crossings` status "eval-only ? wire to prod"; served Gate A is now wired, contradicting the doc's own §1.
7. **RALLY_DETECTION_GAP_CLOSURE.md:56** ? `shuttle\_player\_colocation` tagged `[design]`; it's built (high severity; the rally-END rule that consumes it remains design, but the feature is built).
8. **RALLY_DETECTION_GAP_CLOSURE.md:61** ? reuse map calls it "(new feature)"; built.

The correct framing in every case is: **built in `fusion\_features.py`**, but default-OFF + unmeasured on golden ? never "in production".

No staleness found in

- **ANALYZERS_AND_RECONCILERS.md** ? this doc is explicitly "DESIGN ? NOT started" and its signal references are accurate: it correctly describes `net\_crossings`/person features as the already-built prototype analyzers the future framework retrofits, and correctly tags shuttle-state, warm-up, service-fault, and the reconcilers as not-yet-built design. The `\_enrich\_candidate\_stats` (`fusion\_hybrid.py:76`) and `experiment.predict`/`\_gate\_mask` (treating `net\_crossings`/`players\_in\_court` as columns, `experiment.py:282-290`) references are correct. (Some internal `:LINE` anchors have drifted, e.g. `experiment.py:213` now points to `load\_golden\_videos` not `predict`, but line-number drift is outside this slice's signal-build-status scope.)
- **ABLATION_LAYER.md** ? accurate: `gate\_a`/person/audio/shuttle columns and the Gate-A `structural`/litmus claims all match `ablation.py`.

The Gate-A litmus numbers (`+0.074`, 6W/0L, n=6 corpus) in ABLATION_LAYER.md are framed as the persisted 6-golden-JSON substrate, which is consistent with `ablation.py`'s corpus loader ?

distinct from the n=13 windowing-only corpus in the gap-closure doc, so not a contradiction.